

Real-time Targeted Speech Enhancement

Presented by:

Abdelrahman Adel Abdelfattah

Karim Alaa MoheyElDin

Mohamed Magdy El Sayed

Youhanna Yousry Fawzi

Youssef Magdy Helmy

Supervised by:

Prof.Dr. Layla Abo Hadid Abdallah

Alexandria University

Faculty of Engineering

Computer and Systems Department

Abstract

Speech Enhancement (SE) has undergone significant advancements in recent years, moving from traditional algorithmic methods to more advanced deep learning-based models. These deep learning models have greatly improved speech quality and intelligibility. A major development in SE is personalized speech enhancement (PSE), where models are trained to enhance the speech of specific speakers. This development was evident in the DNS Challenge 2023, where the top-performing models were personalized [1]. Our goal is to further enhance the challenge by introducing real-time constraints, which will impose additional limitations on the model’s architecture and size.

In this work, we explore various models and architectures in SE. We propose two models: the Personalized Fast FullSubNet (PFFSN) and the Personalized Denoiser (PDenoiser). Both models build on established SE architectures. We extend these models by adding personalization support. PFFSN, an adaptation of FFSN [2], uses a subband approach to classify and amplify the subbands where the targeted speaker’s voice resides. PDenoiser, an adaptation of Denoiser [3], suppresses both stationary and non-stationary noises as well as non-primary speakers, all while operating in the time domain. Our experiments demonstrate that our models can compete with state-of-the-art models in terms of speech quality and intelligibility, as evidenced by its performance in the DNS Challenge.

We also introduce a novel Arabic dataset specifically designed for personalized speech enhancement, which was meticulously collected independently. This dataset comprises 30 speakers, with a demographic distribution of 73% male and 27% female participants. The training-validation set includes 30,000 audio clips amounting to a total of 83 hours of recorded speech, while the test set encompasses 1,200 audio clips totaling 3.3 hours.

Finally, we are pleased to introduce a real-time desktop application that seamlessly integrates trained models. This application features a user-friendly graphical interface (GUI) for effortless operation and supports on-device real-time audio processing. It incorporates optimizations for streaming procedures, including adjustable frame size and multi-threading. Impressively, it achieves a low real-time factor (RTF) of 0.21.

Contents

1	Introduction	6
2	Problem Definition	8
2.1	Speech Enhancement	8
2.2	Speech Dereverberation	9
2.3	Audio Source Separation	9
2.4	Music Source Separation	10
2.5	Speaker Embedding	10
2.6	Speaker Verification	11
2.7	Speaker Diarization	11
2.8	Speech Separation	11
2.9	Target Speech Extraction	12
2.10	Our Task	12
3	Background	13
3.1	Audio Representation [20]	13
3.1.1	Waveforms	13
3.1.2	Input Representation	14
3.1.2.1	Time-Frequency (TF) Representation	14
3.1.2.2	Short-time Fourier Transform (STFT)	15
3.1.2.3	Mel-spaced Spectrograms	18
3.1.3	Output Representation	18
3.2	Deep Learning	19
3.2.1	Convolution Neural Networks (CNNs) [20]	19
3.2.1.1	Convolutional Layers	19
3.2.1.2	Key Parameters of CNNs	19
3.2.1.3	CNNs in Audio Processing	20
3.2.2	Recurrent Neural Networks [22] [23]	20
3.2.3	Long Short-Term Memory	22
3.2.3.1	LSTM's Internal Mechanism	24
3.2.3.2	Bidirectional LSTM	25
3.2.4	Multi-Head Self-Attention (MSA)	25
3.2.4.1	Attention	25
3.2.4.2	Scaled Dot-Product Attention	26
3.2.4.3	Multi-Head Attention	27
3.2.5	UNet Architectures	28
3.2.5.1	Encoder	28

3.2.5.2	Decoder	28
3.2.5.3	Skip Connections	28
3.2.5.4	Speech Enhancement with U-Net	28
4	Datasets	30
4.1	Valentini [27]	30
4.1.1	Training Dataset	30
4.1.2	Testing Dataset	31
4.2	DNS [1], [29], [30]	31
4.2.1	DNS 2020 [29]	31
4.2.1.1	Clean Dataset	31
4.2.1.2	Noise Dataset	31
4.2.1.3	Noisy Dataset	32
4.2.2	DNS 2022 [30]	32
4.2.2.1	Clean Dataset	32
4.2.2.2	Noise Dataset	33
4.2.2.3	Impulse Response	33
4.2.3	DNS 2023 [1]	33
4.2.3.1	Challenge Dataset	34
4.3	LibriSpeech	34
4.3.1	Training Set	35
4.3.2	Development Set	35
4.3.3	Test Set	35
4.4	Collected Dataset	35
4.4.1	Motivation	35
4.4.2	Data Collection	36
4.4.2.1	Train-validation split	36
4.4.2.2	Test split	36
5	Evaluation Metrics	37
5.1	Objective Metrics	37
5.1.1	Signal-to-Noise Ratio (SNR)	37
5.1.2	Perceptual Evaluation of Speech Quality (PESQ)	37
5.1.3	Short-Time Objective Intelligibility (STOI)	38
5.1.4	Mean Opinion Score (MOS)	38
5.1.5	DNSMOS P.808 [34]	39
5.1.6	DNSMOS P.835 [35]	40
5.1.7	Real-time Factor (RTF)	41
5.1.8	Word Error Rate (WER)	41
5.2	Subjective Metrics	42
5.2.1	Subjective Mean Opinion Score (SMOS)	42
6	Related Work	43
6.1	DEMUCS	43
6.1.1	Model Architecture	43
6.1.2	Experimental Setup	44
6.1.2.1	Dataset	44
6.1.2.2	Training	45
6.1.3	Conclusion	45

6.2	CleanUNet	45
6.2.1	Model Architecture	46
6.2.2	Experimental Setup	47
6.2.2.1	Datasets	47
6.2.2.2	Training	48
6.2.3	Conclusion	48
6.3	VoiceFilter	49
6.3.1	Model Architecture	49
6.3.1.1	Speaker Encoder	50
6.3.1.2	VoiceFilter	50
6.3.2	Experimental Setup	50
6.3.2.1	Dataset	50
6.3.2.2	Evaluation	51
6.4	VoiceFilter-Lite	52
6.4.1	Model Architecture	52
6.4.1.1	Model Quantization	53
6.4.2	Experimental Setup	53
6.4.2.1	Model and Data	53
6.4.3	Conclusion	53
6.5	Inter-Subnet	54
6.5.1	Model Architecture	54
6.5.1.1	Subband units	54
6.5.1.2	SubInter module	55
6.5.2	Experimental Setup	55
6.5.2.1	Dataset	55
6.5.2.2	Training	55
6.5.3	Conclusion	56
6.6	Fast FullSubNet	56
6.6.1	Model Architecture	57
6.6.1.1	Encoder	57
6.6.1.2	Bottleneck	58
6.6.1.3	Decoder	58
6.6.2	Experimental Setup	58
6.6.2.1	Dataset	58
6.6.2.2	Training	59
6.6.3	Conclusion	59
6.7	TitaNet	59
6.7.1	Model Architecture	59
6.7.1.1	Encoder	59
6.7.1.2	Decoder and Embeddings	60
6.7.1.3	Loss funciton	61
6.7.2	Experimental Setup	61
6.7.2.1	Dataset	61
6.7.3	Conclusion	61
6.8	ECAPA-TDNN	61
6.8.1	Model Architecture	62
6.8.1.1	Context-dependent statistics pooling	62
6.8.1.2	Res2Blocks	62

6.8.1.3	Hierarchical feature aggregation	63
6.8.2	Experimental Setup	63
6.8.3	Conclusion	64
7	Models	65
7.1	Project Stack	65
7.1.1	PyTorch	65
7.1.2	PyTorch Lightning	66
7.1.3	Hydra	66
7.1.4	Weights and Biases	67
7.1.5	TensorBoard	67
7.2	Previous Experiments	68
7.2.1	CleanUNet	68
7.2.2	Denoiser	68
7.2.3	VoiceFilter	69
7.3	Personalized Fast FullSubNet	70
7.3.1	Motivation	70
7.3.2	Speaker Embedding Module	70
7.3.3	Denoising Module	71
7.3.4	Experimental Setup	71
7.3.5	Results	72
7.3.6	Limitations	72
7.4	Personalized Denoiser	72
7.4.1	Motivation	72
7.4.2	Speaker Embedding Module	73
7.4.3	Denoising Module	73
7.4.4	Experimental Setup	73
7.4.5	Results	74
7.4.6	Limitations	74
7.5	Model Selection	75
8	Application	76
8.1	App Features	76
8.1.1	Graphical User Interface: Tkinter and CustomTkinter	77
8.1.2	Database Management: SQLite	77
8.1.3	Technical Details	77
8.2	Streaming	78
9	Future Work	82
	References	83

Chapter 1

Introduction

Speech enhancement is a critical domain within signal processing, dedicated to refining speech signals captured under adverse conditions marked by noise or degradation. The core objective is to enhance the quality and intelligibility of these speech signals. The Real-time Targeted Speech Enhancement project aims to address the challenge of strengthening speech signals in a real-time concerning clear speech provided by the user.

The overarching aim of speech enhancement is to elevate the clarity and understandability of speech, particularly in challenging environments. In many applications, a fundamental requirement for a speech enhancement system is its ability to operate in real-time with minimal latency, preferably on cost-effective hardware integrated into communication devices. The quality of speech in virtual interactions significantly impacts user experience and effective communication.

This document presents an in-depth exploration of the problem definition, background information, related work, proposed models, the application and future work.

Chapter 2 focuses on the problem definition, outlining the core objectives of the project and the challenges associated with speech enhancement in the context of online communication. The chapter also highlights the importance of leveraging deep learning methods specifically tailored for real-time speech enhancement.

Chapter 3 delves into the fundamental concepts of audio representation, including waveforms, time-frequency representations, and output representations. The chapter also explores the application of deep learning methodologies, such as Convolutional Neural Networks and Recurrent Neural Networks, in speech enhancement and audio processing.

Chapter 4 discusses the datasets used in the project, including the Valentini dataset, the DNS datasets, and the LibriSpeech dataset and our own collected dataset. The chapter also provides an overview of the data augmentation techniques used to increase the size of the training data.

Chapter 5 presents the evaluation metrics used to assess the performance of the speech enhancement models including both objective and subjective metrics. It introduces the Perceptual Evaluation of Speech Quality (PESQ), the Short-Time Objective Intelligibility (STOI) metrics, Deep Noise Suppression Mean Opinion Score (DNSMOS) and other metrics used in the evaluation of speech enhancement models.

Chapter 6 offers a comprehensive review of the latest advancements in speech enhancement, focusing on the state of the art and identifying gaps in the current research. This chapter also explores various speaker embedders and their integration into cutting-edge architectures. Furthermore, it emphasizes the unique contributions and potential impact of the Real-time Targeted Speech Enhancement project on the speech enhancement domain.

Chapter 7 focuses on the proposed models developed as part of the project, including the Personalized Fast FullSubNet (PFFSN) and the Personalized Denoiser (PDenoiser). The chapter provides an in-depth analysis of the models' architectures, training procedures, and evaluation results. It also discusses the performance of the models on the collected dataset and the DNS Challenge 2023.

Chapter 8 provides an overview of the application developed as part of the project, detailing its primary components and functionality. The chapter also discusses the workflow of the application, divided into two main stages: the GUI app and the streamer. The GUI app serves as the interface for user interaction, capturing user inputs and passing them to the subsequent stages of the application. The streamer handles real-time audio input, utilizing a Virtual Audio Cable (VB cable) to capture the user's voice. The chapter also provides a detailed explanation of the application's streamer and features.

Chapter 9 outlines potential future directions for the project, including the integration of additional features, optimization of the existing models, and the exploration of new datasets. The chapter also discusses the potential for expanding the application's functionality and enhancing its real-time capabilities.

Chapter 2

Problem Definition

A wide range of tasks related to speech processing exist. They revolve around manipulating and operating on spoken language signals [4]. The main challenge is to address the difficulties introduced in real-world conditions. In real-world conditions, noise and distortions take place, and fighting these conditions is the main goal of speech enhancement. Previously, algorithmic methods were used while now shifting towards deep learning-based algorithms to improve the speech signal. A related task is speech dereverberation, which removes the reverberant effect caused by enclosed areas.

Another branch of tasks is audio separation tasks, where the goal is to extract specific or targeted audio sources from a mixed audio signal. This has several applications, not only in speech but also in music. Music source separation is a common application, where the goal is to extract specific sound sources such as vocals and musical instruments. Another vital example is speech separation, where the mixed signal consists of multiple speakers and the goal is to extract the voice of each speaker. It can also be targeted for a specific speaker voice, i.e., extracting that target voice exactly which is known as target speech extraction.

Other speech-related tasks focus on operating on speech instead of manipulating it. Speech recognition translates the spoken speech into text. Speaker recognition and diarization focus on identifying and tracking speakers in the given signal. Speech embeddings attempt to convert the signal into low-dimensional vector representations, which allows tasks such as speaker verification and sentiment analysis to exist.

2.1 Speech Enhancement

Speech Enhancement is a signal processing task that involves improving the quality of speech signals captured under noisy or degraded conditions. Speech enhancement aims to improve the clarity, intelligibility, and listenability of speech signals, which can be employed for voice recognition, teleconferencing, and hearing aids, among other applications.

The majority of recorded conversational speech signals have some kind of background noise that makes it difficult to understand, including tapping on a keyboard, dogs barking, or traffic noise. Therefore, improving speech is a crucial challenge in and of itself for hearing aids [5], audio and video calls [6], and automatic speech recognition (ASR) systems. In many of these applications, a fundamental requirement for a speech enhance-

ment system is its ability to operate in real-time with minimal latency (online), preferably on cost-effective hardware integrated into communication devices.

In this task, the input audio signal $i \in \mathbb{R}^T$ can be described as the summation of two signals, the clean speech signal $c \in \mathbb{R}^T$ and an additive noise signal $n \in \mathbb{R}^T$ where T is the length of the signal. Meaning that $i = c + n$. The goal of this task is to find an enhancement function f such that $f(i) \approx c$ [3].

The significance of speech enhancement lies in its ability to improve speech signals in situations when other sources of interference, reverberation, or background noise degrade speech signals. In telecommunications, for instance, speech enhancement techniques are crucial for improving the quality of voice calls in noisy environments, ensuring seamless communication even in challenging conditions. Speech enhancement can be linked to audio separation tasks. However, they differ in their goals. Speech enhancement is only interested in getting an approximation of the clear speech signal, in contrast with audio separation which aims to disentangle and extract specific audio sources. The problem of Speech enhancement can be constructed as a source separation problem, with one of the sources always being a speaker and the other always being noise (the "noise" source can be modeled or disregarded entirely).

2.2 Speech Dereverberation

Reverberation is an audio disturbance that arises in enclosed areas due to repeated reflections and diffractions of sound off objects and walls. These numerous echoes amplify the direct sound and distort the temporal and spectral qualities of a speaker in a room. By placing a microphone close to the source of the relevant signal, its effect can be mitigated. But this isn't practical for "hand-free" applications, like men-machine communication, for example. A lot of applications that call for a distant sound pickup struggle when reverberation is present. This is true for automatic speaker verification and automatic speech recognition. Since reverberation can make speech less understandable, dereverberation can also be beneficial to listeners with hearing impairments [7]. Speech dereverberation can be considered a subtask of speech enhancement since the reverberation effect can be considered as a part of the noise signal that the enhancement function is trying to eliminate.

2.3 Audio Source Separation

Audio source separation tasks are interested in the separation of a mixture of sounds into a set of separate sounds [8]. These separate sounds are often from distinct sources. These sources can be humans, targeted humans, or musical instruments. In this task, we can say that given an input mixture signal $y(t)$ that is composed of N sources, $x_n(t)$, for $n = 1..N$, such that $y(t) = \sum_{i=1}^N x_i(t)$. The goal of this task is to recover one or more $x(t)$.

In this task, it is common for the mixture to have more sources than signals. As a result, source separation is considered an underdetermined problem, which means that there are fewer observations (the mixture) than needed outcomes (the required source(s)).

There are different variations of this task which will be elaborated in the next sections.

2.4 Music Source Separation

Music source separation is one of the most popular examples of audio source separation. It aims to separate a complicated audio mixture into its component signals, isolating specific sound sources such as vocals, musical instruments, or other sound sources. The extraction of separate audio sources from a composite audio signal presents challenges and complications that cross multiple scientific fields.

Music is considered a distinct problem from other types of source separation due to a variety of factors that make it notably difficult [9]:

1. In music, sources are highly correlated, which means that all the sources frequently change at the same time.
2. Music is combined and processed in non-linear and physical ways. Because of modern recording techniques, any particular source in a mixture may never exist naturally in the real world. This is problematic since we rarely, if ever, know what processing was used on each source or the entire mixture.
3. Usually, the output of a music source separation system is employed in an end-user system making the quality bar significantly higher.

2.5 Speaker Embedding

Speaker embeddings play a crucial role in converting a speech signal into a low-dimensional vector representation. This compressed format captures the essence of a speaker's characteristics, enabling various speech-processing tasks. It lays the groundwork for applications like speaker verification and sentiment analysis. The process of using speech embeddings can be broken down into several steps. First, a raw speech signal is fed as input. This signal is then processed by an embedding model, typically a deep neural network trained to analyze speech and extract speaker-specific information. The model then generates a fixed-length vector representation, also known as the embedding. This embedding vector essentially encodes speaker characteristics such as pitch, timbre, and speaking style. Interestingly, embeddings for similar speakers will reside closer together in a vector space, whereas embeddings for distinct speakers will be further apart [10]. Speech embeddings open doors to a variety of speech-processing applications. One prominent example is speaker verification. In this task, the system determines whether a speaker's voice matches a claimed identity. This is achieved by comparing the embedding of the claimed speaker with the embedding of a captured voice sample. Another application is speaker diarization [11]. Here, the goal is to segment an audio recording into sections and identify the speaker in each segment. By analyzing the embeddings throughout the recording, the system can achieve this diarization task. Speech embeddings can even be incorporated into sentiment analysis. In this case, the emotional tone of speech is analyzed by considering both the speech content and the speaker embeddings. By learning and representing speaker characteristics in a compressed format, speech embeddings offer a powerful tool for various speech-processing tasks. This technology paves the way for advancements in applications that rely on understanding the who and the how behind spoken language.

2.6 Speaker Verification

Speaker verification is the process of verifying the identity of the speaker according to the characteristics of their voice.

There are two types of speaker verification [12]: text-dependent and text-independent. Text-dependent methods assume that the user will speak the text that has been pre-programmed into the speaker verification system. On the other hand, in a text-independent system, the user is not expected to cooperate and the system does not have any prior knowledge about the text to be spoken.

2.7 Speaker Diarization

Speaker diarization involves dividing audio recordings into segments labeled with speaker identities, addressing the question of "Who spoke when?" This process contrasts with speech recognition, which focuses on transcribing spoken content.

Speaker diarization and speaker identification share common ground in their approaches to analyzing audio signals for speaker-related information. Both tasks involve segmenting audio recordings to discern speaker boundaries and characteristics. Speaker diarization focuses on partitioning the entire recording into segments associated with different speakers without prior knowledge of their identities, addressing the question of "who spoke when?" Speaker identification, on the other hand, aims to identify specific speakers within shorter segments of audio, assuming the number of speakers is known or can be determined. Both tasks rely on extracting features such as pitch, timbre, and prosody from the audio, and utilize machine learning techniques including clustering, Gaussian mixture models, and deep learning approaches. These tasks are essential in applications such as automated transcription, surveillance, and speaker verification systems, where understanding both the structure of conversations (diarization) and the specific identity of speakers (identification) are crucial for accurate analysis and decision-making.

2.8 Speech Separation

Speech separation is by far the most common example of audio source separation. Speech separation's main goal is to separate the voices of 2 or more people who are talking at the same time.

The human hearing system is extraordinary in its capacity to perceive speech even in noisy situations [13]. To focus on the speaker we want to hear, we effortlessly filter away competing voices. This problem, known as the cocktail party problem led to the creation of advanced voice separation algorithms [14].

Speech separation is not straightforward for machines to replicate [15]. It has its challenges to overcome [14]:

1. When multiple people speak at the same time, their voices overlap and conceal each other, making it difficult to distinguish individual words. This is the main problem of speech separation, and it involves the use of algorithms to separate the overlapping audio streams.
2. Speech signals are dynamic, with pitch, intensity, and timbre changing all the time

- [16]. Because of this non-stationarity, algorithms find it challenging to adjust and accurately distinguish various voices, especially when they share similar features.
3. Real-world recordings frequently include background and ambient noise such as traffic, machinery, or music, which further distort the target voice [17]. Algorithms must correctly recognize and filter these disturbances while not interfering with the target speech.
 4. Speech separation algorithms must be resilient to changes in speaker characteristics such as gender, age, accent, and language. A model trained on one particular type of speaker might not operate effectively with different types, making generalized approaches difficult to develop.

Several separation algorithms and techniques were developed for speech initially and then adopted into other fields, like music separation.

2.9 Target Speech Extraction

Speech extraction is the task of extracting human speech from a given audio signal. A variation of it is the target speech extraction (TSE) task, which extracts only the speech of a target voice.

Handling interfering speakers is a key issue since target and non-target speech signals have similar features, making detection difficult. TSE isolates a target speaker’s speech signal from a combination of numerous speakers with or without noises and reverberations. This makes it overlap with the speech enhancement/denoising task.

TSE can be possible by employing clues that identify the speaker in the mixture. A spatial indication showing the direction of the target speaker, a video of the speaker’s lips, or a pre-recorded enrollment utterance from which their voice characteristics can be inferred are examples of such clues [18].

TSE is a new topic of study that has gained popularity in recent years since it provides a practical solution to the cocktail party problem [19] and incorporates signal processing elements such as audio, visual, array processing, and deep learning.

2.10 Our Task

Our specific focus lies in addressing the challenges associated with speech enhancement in the context of online communication. This is a crucial area, as the quality of speech in virtual interactions significantly impacts user experience and effective communication. Our proposed approach involves leveraging Deep Learning methods specifically tailored for Real-Time Speech Enhancement. It is noteworthy that real-time implementations, by nature, demand heightened efficiency compared to their non-real-time counterparts. In our pursuit of advancing the current state-of-the-art, we aim to enhance an existing speech enhancement method. We plan to achieve this by incorporating a crucial element—utilizing a reference of **clear speech** provided by **the device user**. This reference will serve as a valuable asset, aiding the model in effectively extracting and enhancing the user’s voice even in the presence of substantial ambient noise. This innovative approach is anticipated to contribute significantly to the improvement of speech quality in online communication scenarios.

Chapter 3

Background

3.1 Audio Representation [20]

Starting the examination of input and output representations fundamentals to speech enhancement systems, we turn our attention to the fundamentals of audio data. A waveform is the most basic type of audio, which is generated when a microphone detects air pressure fluctuations and converts them into an electrical signal. While some speech improvement approaches work directly on this waveform, a significant share requires preparatory pre-processing processes before engaging in substantive noise reduction procedures. This section carefully examines the many types of input and output representations used in speech enhancement techniques.

Our research begins with a thorough examination of waveforms, which are regarded as the most fundamental representation of audio. Following that, we examine commonly used audio representations that are specifically designed for speech enhancement applications.

3.1.1 Waveforms

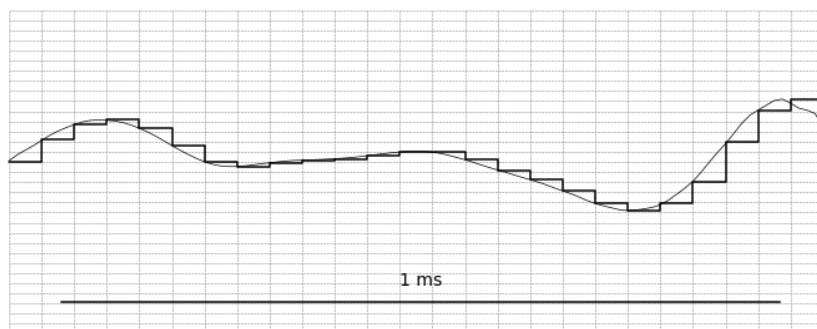


Figure 3.1: *Each value within the waveform is sampled at a consistent rate and precisely quantized, enabling accurate digital representation of the audio.*

Waveforms are the graphical representations of sound. Similar to ripples in a pond, sound waves are captured by a microphone and converted into an electrical signal. This signal

undergoes segmentation (sampling) and quantization processes before materializing as the waveform visible on audio visualization platforms. The distinction between mono and stereo waveforms lies in the number of channels: a singular line for mono and dual lines for stereo.

Considering the recreation of an image by tracing outlines with dots, an analogy with audio recordings may be established to highlight the importance of sampling rate. The complexities of sample rate relate to the frequency of sound wave sampling each second, measured as Hertz (Hz). Higher sample rates allow for more detailed capture, particularly of high-pitched sounds. The Nyquist frequency, which represents half the sample rate, defines the highest limit for reliably recorded frequencies. Frequencies above the Nyquist frequency, sometimes known as the "sound ceiling," may cause distortion.

Additionally, downsampling, which involves reducing the sample rate to improve computational efficiency, is analogous to utilizing fewer dots in an image trace. While downsampling can speed up model training, careful consideration is required. An excessive reduction in sample rate risks deleting critical high-frequency components, thereby compromising the clarity and integrity of the enhanced speech output. As a result, choosing an optimal downsampling rate is critical for achieving a balance between computing efficiency and the preservation of critical audio data.

3.1.2 Input Representation

3.1.2.1 Time-Frequency (TF) Representation

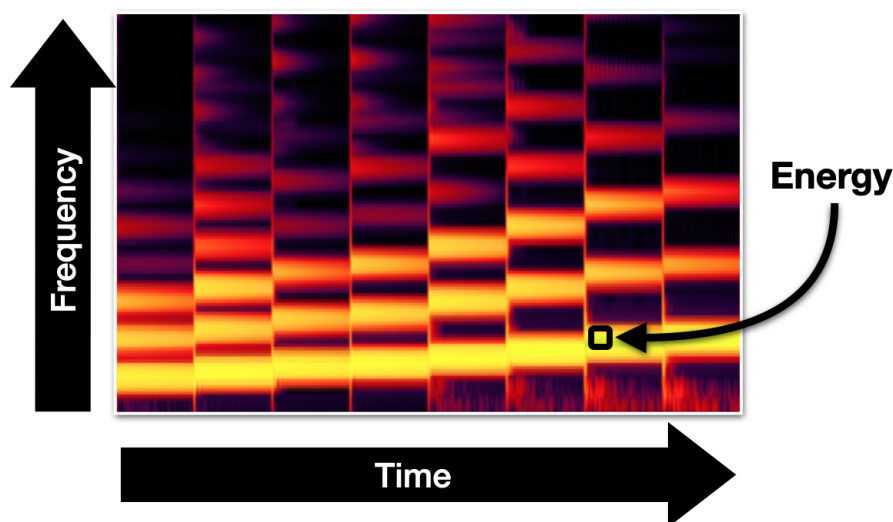


Figure 3.2: *A time-frequency representation. Time runs along one axis, frequency along the other. Each point's brightness reveals the sound's energy at that specific time and frequency.*

The TF representation can be thought of as a two-dimensional map, where one axis tracks time and the other maps frequency like the different notes played. Each tiny square on the map shown in Figure 3.2, called a "TF bin," captures the energy (loudness) of the sound at that precise moment and frequency.

It can be visualized as a heatmap, where brighter colors usually hint at louder sounds, while darker shades whisper quieter ones. Some heatmaps even have a color bar, like a legend, to clearly explain which colors correspond to which sound levels. So, by peering into this colorful landscape, we can learn how frequencies change and shift over time, unlocking valuable insights into the inner workings of sound.

3.1.2.2 Short-time Fourier Transform (STFT)

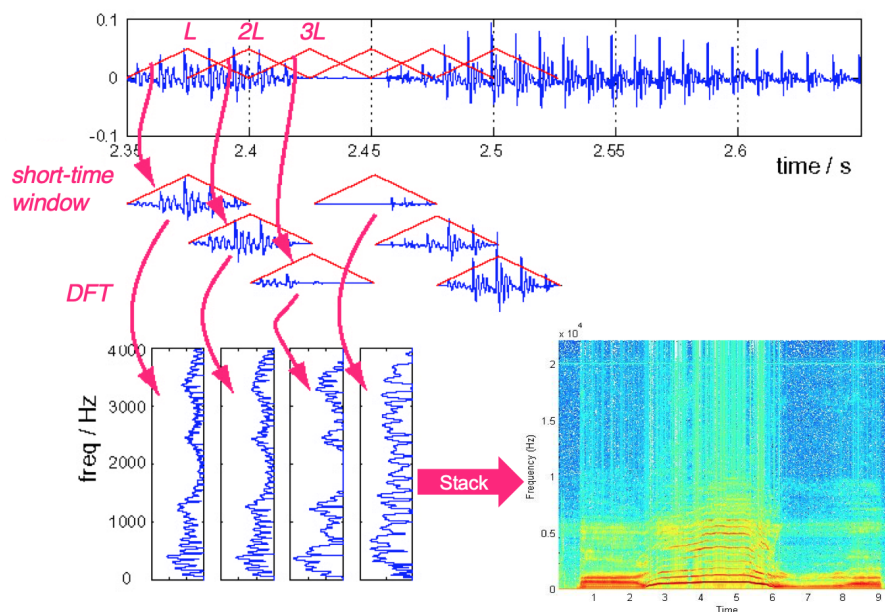


Figure 3.3: *Computation process of short-time Fourier transform of waveform. Image by Bryan Pardo.*

The Short-time Fourier Transform (STFT) plays a crucial role in speech enhancement, providing a powerful lens through which we can dissect and manipulate audio signals.

To transform a continuous audio waveform into an STFT representation, we follow these steps:

1. **Dividing the Waveform:** The audio is first divided into a series of short, overlapping segments.
2. **Applying the DFT:** For each segment, a mathematical operation called the Discrete Fourier Transform (DFT) extracts the individual frequencies present within that segment.
3. **Constructing the Time-Frequency Matrix:** The STFT assembles these frequency snapshots into a two-dimensional matrix, creating a visual representation of how frequencies change over time.

Each element within this matrix, called a TF bin, holds complex-valued information, encompassing both magnitude and phase:

- **Magnitude:** Represents the strength or energy of a particular frequency at a given time.

- **Phase:** Encodes the precise timing relationships between different frequencies.

The STFT is a reversible transformation, meaning we can reconstruct the original audio waveform from its STFT representation using a process called the inverse STFT (iSTFT). In speech enhancement applications, the STFT's ability to decompose audio into its constituent frequencies over time proves invaluable for identifying and manipulating noise patterns, ultimately leading to clearer and more intelligible speech.

When computing STFT there exist multiple parameters to take into consideration here are some of them:

- **Window Types:** When analyzing audio for speech enhancement, the window type acts like a lens, shaping how frequencies are captured in short segments before the DFT. Different window shapes can highlight or downplay certain frequencies, influencing the final STFT representation. Figure 3.4 shows the most widely used windows when calculating STFT.
- **Window Length:** Window length dictates how many samples of audio are captured within each short segment before applying the DFT. This choice directly affects the frequency resolution of the resulting STFT representation; the longer the window the higher the frequency resolution and vice versa as shown in figure 3.5.
- **Hop Length:** The hop length dictates the separation in samples, between successive short-time windows. Figure 3.6 shows how adjusting the hop length can either extend or compress the time axis in a STFT.
- **Overlap-Add:** When converting a signal to an STFT and back to a waveform, certain parameter values for window type, window length, and hop length may not precisely recreate the original signal. A certain set of parameters, on the other hand, is theoretically guaranteed to properly reconstruct any signal. These parameter sets are known as Constant Overlap-Add (COLA) because they add up to a constant value when subsequent windows are applied.

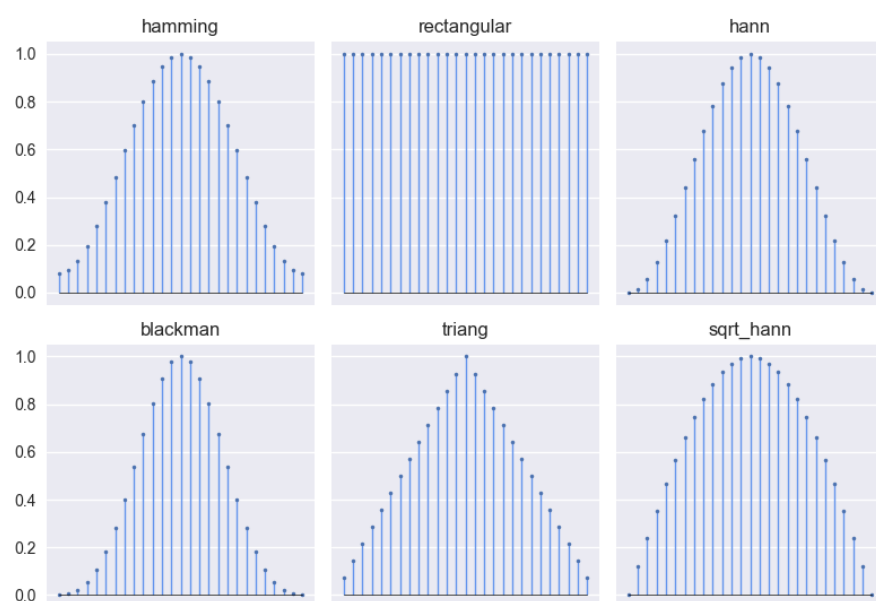


Figure 3.4: Six widely used window types for computing STFT.

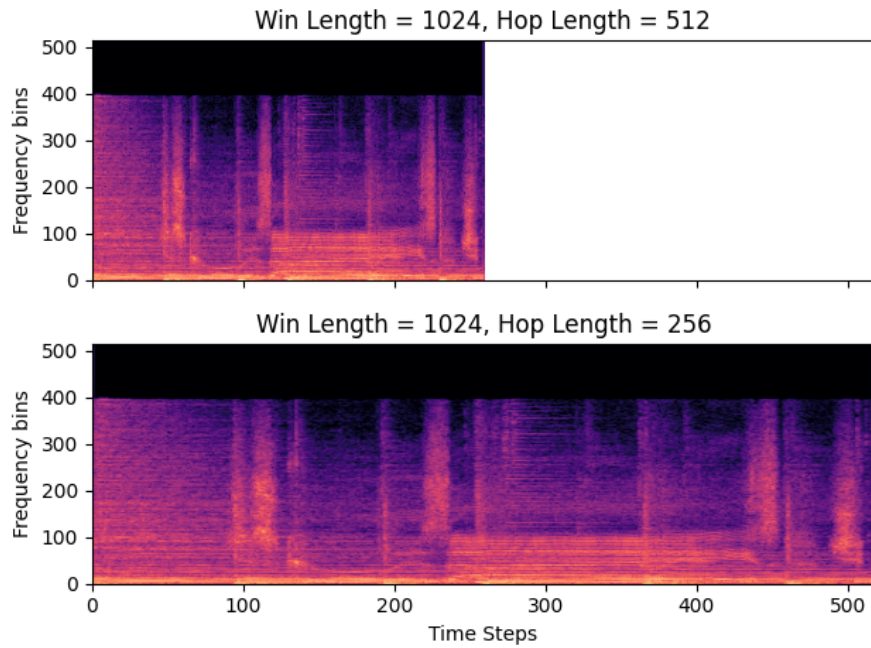


Figure 3.5: *Window length represents the number of samples taken per window. A trade-off between the window length and the frequency resolution is shown.*

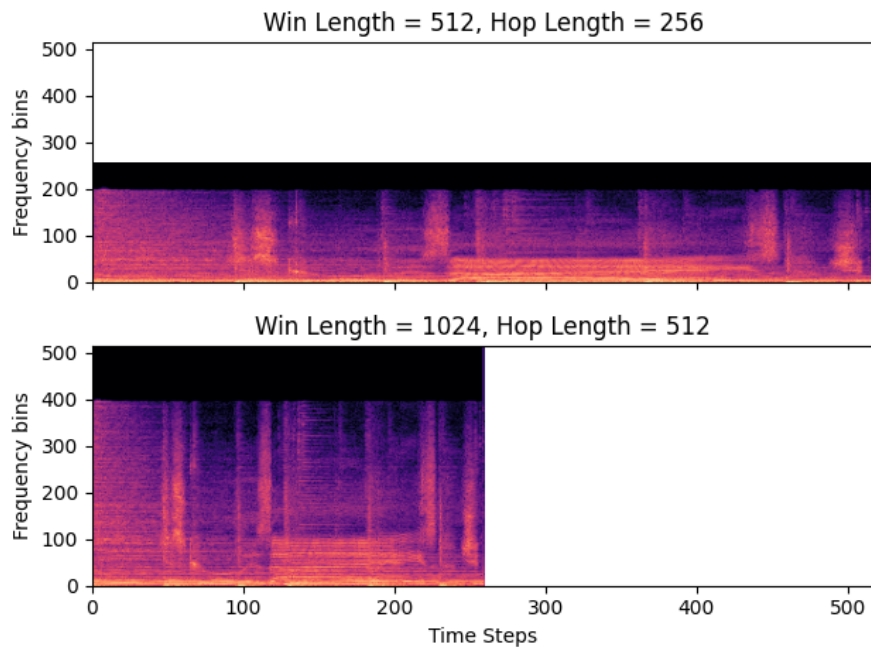


Figure 3.6: *The hop length determines the distances in samples between contiguous short-time windows. The smaller the hop length the more times a portion of the audio signal is present in the STFT.*

3.1.2.3 Mel-spaced Spectrograms

In terms of frequencies, human hearing is logarithmic! The Mel scale approximates this characteristic and is a simple technique to create a spectrogram's frequency axis. Because the number of Mel-spaced frequency bins is generally less than the number of linearly-spaced frequency bins, this is also commonly used to reduce the computational load on deep learning-based techniques.

Figure 3.7 shows how the Mel-spectrogram compresses higher frequencies and leaves more space for lower frequencies.

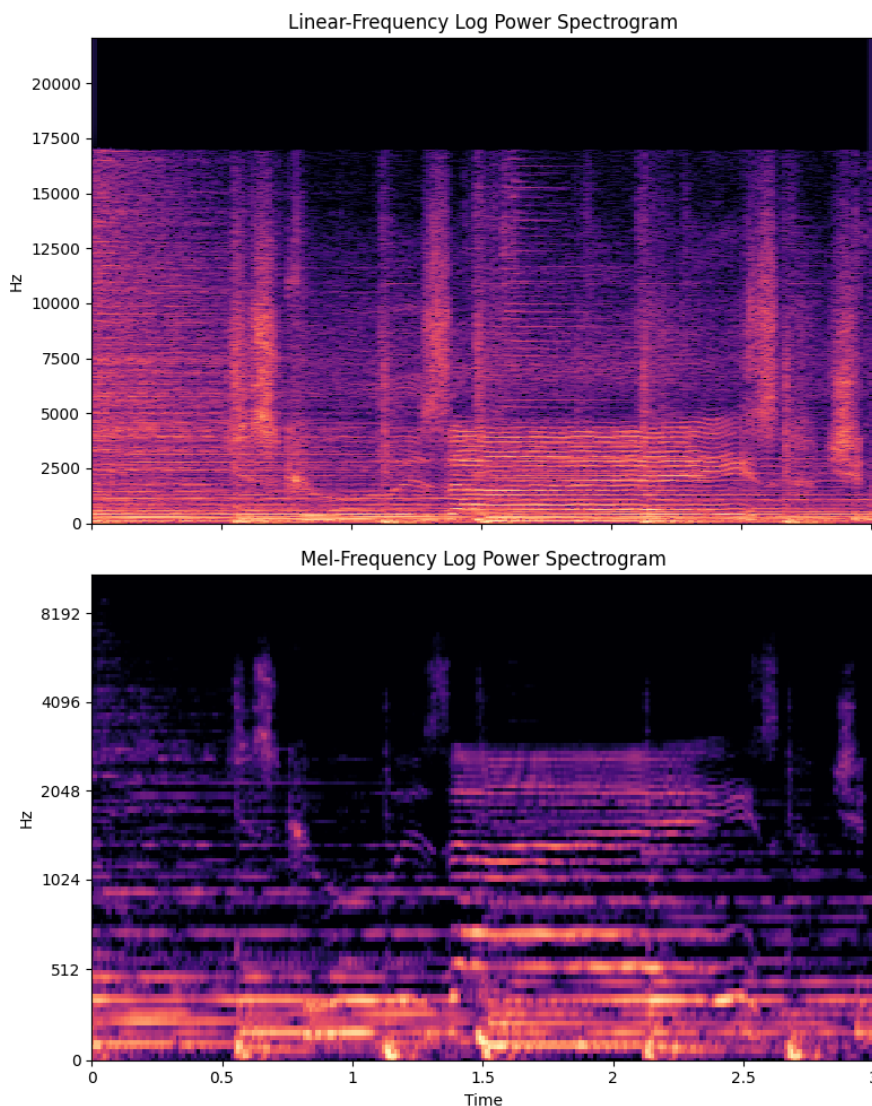


Figure 3.7: *A comparison between linear-scaled and Mel-scaled y-axis.*

3.1.3 Output Representation

While some speech enhancement models directly output the clean audio in waveform other models output a mask that is applied to the spectrogram of the source audio and the result is converted back to a waveform.

3.2 Deep Learning

3.2.1 Convolution Neural Networks (CNNs) [20]

Convolutional Neural Networks (CNNs) are a powerful class of deep learning models renowned for their exceptional performance in image recognition, classification, and other tasks involving grid-like data. Unlike fully connected neural networks, where each node connects to all nodes in the previous layer, CNNs employ a more nuanced architecture inspired by the human visual cortex. This section introduces the key concepts and functionalities of CNNs, highlighting their advantages and potential challenges.

3.2.1.1 Convolutional Layers

The core of a CNN resides in its convolutional layers. These layers consist of nodes that only connect to a small, local region of nodes in the preceding layer. This selective approach offers numerous benefits:

- **Reduced Overfitting:** By limiting connections, CNNs are less susceptible to memorizing training data and more adept at generalizing to unseen examples.
- **Translation Invariance:** CNNs recognize patterns regardless of their location within the input. This invariance is crucial for tasks like object detection, where a cat remains identifiable regardless of its position in an image.
- **Efficient Feature Extraction:** Convolutional layers learn a set of filters (kernels) that slide across the input, detecting specific features like edges, corners, and textures. This feature extraction enables higher-level layers to build upon these basic building blocks for complex pattern recognition.

3.2.1.2 Key Parameters of CNNs

The output of a convolutional layer is influenced by four key parameters:

- **Kernel Size:** Determines the dimension and shape of the window that the layer "sees" (Figure 3.8). Larger kernels capture broader patterns, while smaller kernels focus on finer details.
- **Stride:** Defines how far the kernel moves between adjacent input nodes. Larger strides reduce the output size but may lose information.
- **Padding:** Controls how the input edges are handled. Padding with zeros or mirrored values maintain the output size while mitigating information loss at the edges (Figure 3.9).
- **Dilation:** Introduces spaces between the input nodes within the kernel's receptive field (Figure 3.10). This allows convolutional nodes to perceive larger contexts without increasing kernel size.

Unlike standard convolutional layers that shrink dimensionality, transpose convolutions (also known as deconvolutional layers) upsample the output. This operation is valuable for tasks like image generation and autoencoders.

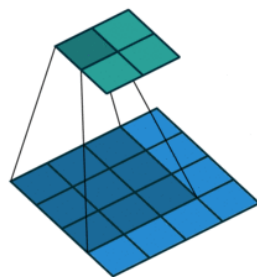


Figure 3.8: *Convolution with 2D 3x3 filter (kernel) applied on 4x4 input and the output is 2x2. [21]*

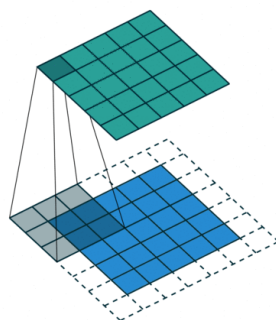


Figure 3.9: *Convolution with 2D 3x3 filter (kernel) applied on 5x5 input with padding 1 and the output is 5x5. [21]*

Pooling layers further reduce the dimensionality of feature maps by applying functions like max-pooling or average-pooling to non-overlapping sections of the input (Figure 3.11). This process retains key information while suppressing redundant details, improving computational efficiency, and preventing overfitting.

3.2.1.3 CNNs in Audio Processing

While image recognition remains a primary domain for CNNs, their application extends to audio processing tasks. However, audio data poses unique challenges, as unlike images, sound waves are sequential and lack predefined shapes. To utilize CNNs in audio, data slicing, and padding may be necessary to conform to the layer expectations. Edge effects can also arise, potentially distorting the predicted waveform. Overlapping windows, similar to Short-Time Fourier Transform (STFT) computations, can alleviate these issues and improve performance.

3.2.2 Recurrent Neural Networks [22] [23]

One fundamental aspect of human cognition is the ability to leverage context when processing information. We understand each word in a sentence not in isolation but concerning those preceding it. Similarly, our comprehension of a movie scene hinges on the accumulated understanding of prior scenes. This inherent "persistence of thought" poses a significant challenge for traditional neural networks, which operate on independent snapshots of data. Their static architecture struggles to incorporate temporal dependencies, hindering their ability to learn from sequential data.

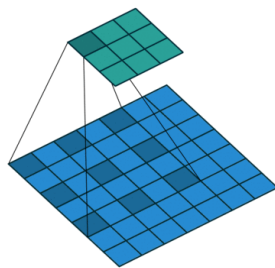


Figure 3.10: *Dilated Convolution with 2D 3x3 filter (kernel) with dilation factor of 1 applied on 7x7 input and the output is 3x3. [21]*

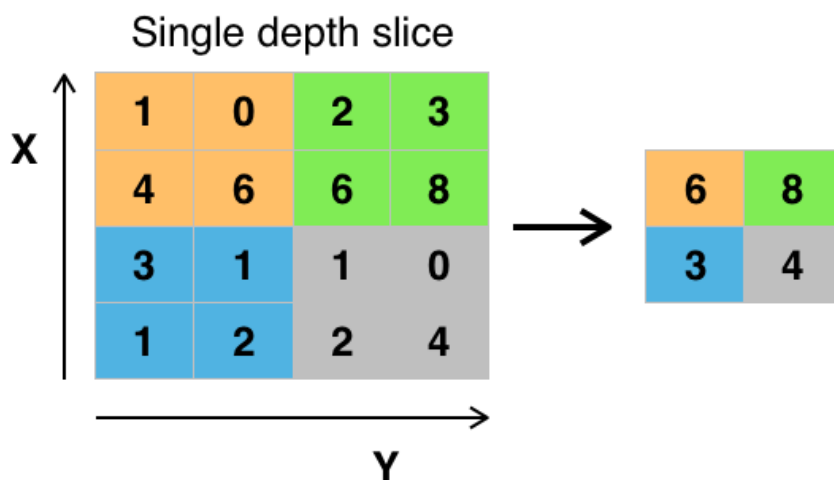


Figure 3.11: *Max pooling operation applied on a matrix. [21]*

Consider the task of classifying events in a film sequence. A conventional neural network analyzing each frame individually would lack the capacity to recognize overarching themes or draw connections across scenes. This shortcoming arises from the network's inability to store and utilize memory, leading to fragmented and context-agnostic interpretations.

Recurrent neural networks (RNNs) overcome this limitation by introducing internal loops that enable information persistence. Unlike their static counterparts, RNNs possess the ability to retain and integrate relevant information from previous input steps, allowing them to build a dynamic understanding of sequences.

This internal memory mechanism can be visualized through the concept of hidden states. As depicted in Figure 3.12, part of the network (denoted by "A") processes an input x_t and generates an output h_t . Crucially, a loop feeds this output back into the network at the next timestep, allowing it to influence the processing of subsequent inputs. In essence, this creates a chain reaction where each step carries forward the accumulated knowledge from previous iterations.

Thinking of an RNN as a chain of interconnected modules clarifies this concept further. Each module receives information from its predecessor, processes it based on its internal state (influenced by past inputs), and generates its output to be passed on. This "message-passing" mechanism allows the network to build a holistic understanding of the

sequence by progressively incorporating context with each step.

The inherent memory capability of RNNs makes them uniquely suited for tasks involving sequential data. They excel in applications like language processing, where understanding sentence structure and semantics requires considering the order of words; speech recognition, where capturing the nuances of spoken language hinges on temporal context; and time series analysis, where predicting future values based on past trends necessitates memory of previous data points.

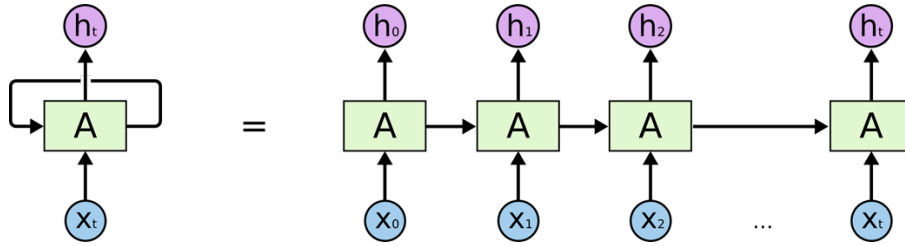


Figure 3.12: *On the left, the looped version of a recurrent layer. On the right, is the unrolled equivalent version of a recurrent layer.*

3.2.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) networks, introduced by Hochreiter and Schmidhuber (1997) [24], stand as a prominent advancement in the field of recurrent neural networks (RNNs). Unlike traditional RNNs, LSTMs possess the ability to effectively learn and utilize long-term dependencies within sequential data, overcoming the vanishing gradient problem that plagued earlier architectures. This section delves into the core principles of LSTMs, exploring their unique structure and the mechanisms that govern their memory capabilities.

While sharing the recurring chain-like structure of conventional RNNs, LSTMs differ significantly in the composition of their recurring modules. Instead of a single tanh layer, LSTMs employ four interconnected layers that interact in a coordinated fashion. As depicted in Figure 3.14, information flows through the network via complete vectors, carried along by lines connecting the output of one node to the inputs of others. Pointwise operations, represented by pink circles, are performed on these vectors, and yellow boxes signify the learned neural network layers. Merging lines denote concatenation while forking lines indicate the copying of content to different locations within the network.

The key element of an LSTM lies in its cell state, symbolized by the horizontal line traversing the top of Figure 3.13. This cell state acts as a conveyor belt, continuously carrying information vertically across the network. Its inherent linearity facilitates the smooth and efficient flow of information, minimizing decay or distortion over time.

LSTMs possess the remarkable ability to selectively add or remove information from the cell state, a process meticulously controlled by specialized components known as gates. These gates, depicted as yellow boxes with sigmoid activation functions and subsequent pointwise multiplication operations, effectively regulate the flow of information through the network. The sigmoid layer outputs values between 0 and 1, determining the degree to which each element is allowed to pass through. A value of 0 signifies complete blockage, while 1 grants unrestricted passage. An LSTM employs three distinct gates to safeguard and manage the cell state:

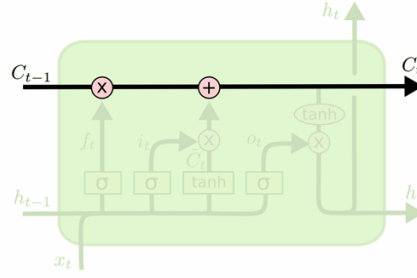


Figure 3.13: Horizontal connection of cell state in LSTM.

- **Forget Gate:** This gate determines which information from the previous cell state should be retained. By outputting values near 0 for irrelevant information and values close to 1 for relevant information, the forget gate effectively filters the cell state, discarding outdated or unnecessary data.
- **Input Gate:** This gate controls the flow of new information into the cell state. It considers both the current input and the previous hidden state, selectively adding relevant information to the cell state based on its output values.
- **Output Gate:** This gate regulates the information that is exposed to the rest of the network. By applying the sigmoid function to the current cell state and the previous hidden state, the output gate determines which portions of the cell state will contribute to the output of the LSTM unit.

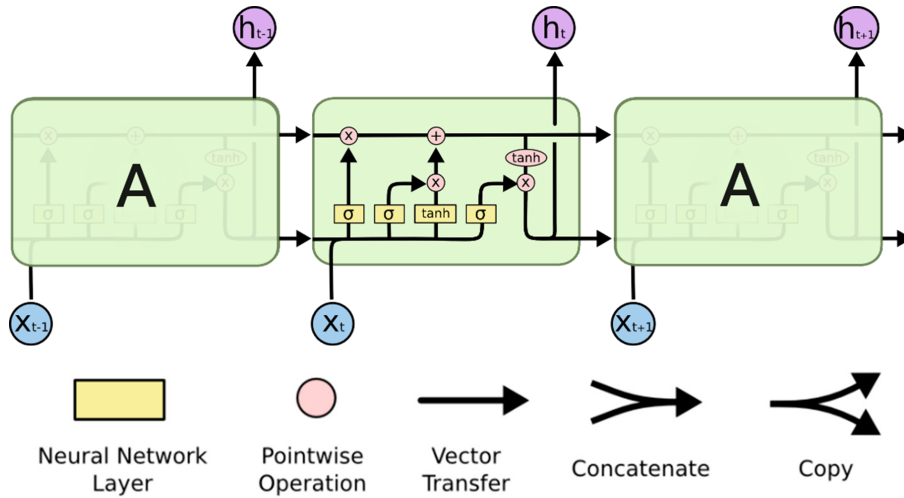


Figure 3.14: Information Flow in a LSTM. Each directed line in the graph corresponds to a complete vector, transmitting information from the output of one node (neuron) to the inputs of others. Pointwise operations, such as vector addition, are denoted by pink circles, while the yellow boxes represent trainable neural network layers. Merging lines indicate vector concatenation, where the elements of two or more vectors are combined to form a single longer vector. Conversely, a forking line signifies the content being copied, with identical copies directed to different locations within the network.

3.2.3.1 LSTM's Internal Mechanism

The first stage involves determining which information to discard from the previous cell state C_{t-1} . This critical decision is facilitated by a "forget gate" implemented as a sigmoid layer. This layer analyzes both the previous hidden state h_{t-1} and the current input x_t to generate a forget vector f_t with values between 0 and 1 for each element of C_{t-1} as illustrated in Equation 3.1. A f_t value of 1 indicates complete retention, while 0 signifies absolute removal. The following equation describes the forget gate's operation:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.1)$$

Consider an LSTM-based language model predicting the next word in a sentence. The cell state might hold information like subject gender for pronoun agreement. When encountering a new subject, the forget gate activates to discard the irrelevant gender information about the previous subject.

The second stage focuses on incorporating new information into the cell state. This process unfolds in two steps. First, an "input gate" sigmoid layer generates an update vector i_t specifying which state elements require modification. Subsequently, a tanh layer creates a candidate update vector $\tilde{C}_t$ containing potential new values as shown in Equations 3.2 and 3.3. These two vectors are then combined to form the state update. The equations governing this stage are as follows:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3.3)$$

Here, the intent is to integrate the gender of the new subject into the cell state, replacing the discarded information about the previous subject. The input gate determines which aspects of the state require an update (e.g., subject gender), while the tanh layer generates potential new values for those elements.

The third stage bridges the gap between the old and the updated cell state C_t . Building upon the decisions made in the previous stages, it implements the actual state update. First, C_{t-1} is multiplied by f_t , effectively discarding the elements marked for removal by the forget gate. Next, the scaled candidate update vector $i_t * \tilde{C}_t$ is added to the filtered C_{t-1} , resulting in the new cell state C_t .

This stage witnesses the actual removal of old subject gender information and the insertion of new information about the current subject's gender, as determined by the forget and input gates in the earlier steps.

While the cell state holds the complete processed information, the output undergoes a selective filtering process. An "output gate" sigmoid layer determines which cell state components are relevant for the current output. Subsequently, the tanh-transformed cell state is multiplied by the output gate vector, ensuring that only the chosen information is emitted.

After processing a subject, the model might want to output information relevant to the upcoming verb, such as its tense and agreement morphology. The output gate decides which aspects of the cell state (e.g., subject number) are crucial for predicting the verb and filters the output accordingly.

By meticulously managing the information flow within the cell state, LSTMs retain long-term dependencies while adapting to new information in sequential data, making them a powerful tool for various applications.

3.2.3.2 Bidirectional LSTM

Bidirectional LSTMs (BiLSTMs) are an extension of traditional LSTMs that can improve model performance on sequence classification problems. They work by processing the input sequence in both directions with two separate hidden layers which are then fed forwards to the same output layer. The motivation behind BiLSTMs is that the output at time t may not only depend on the previous elements in the sequence but also future elements. For example, to predict a missing word in a sequence you want to look at both the left and the right context. The two separate hidden layers of a BiLSTM are typically referred to as the forward and backward layers. The forward layer passes over the sequence from left to right and the backward layer passes over the sequence from right to left. The outputs of these two layers are usually concatenated at each time step, although there are other options, e.g., summation. The output of a BiLSTM at time t is a concatenation of the forward hidden state h_t^f and the backward hidden state h_t^b :

$$h_t = [h_t^f, h_t^b] \quad (3.4)$$

where h_t^f is the hidden state of the forward layer at time t and h_t^b is the hidden state of the backward layer at time t . This bidirectional configuration allows the model to effectively "anticipate" future information by starting from the end of a sequence, such as an audio clip.

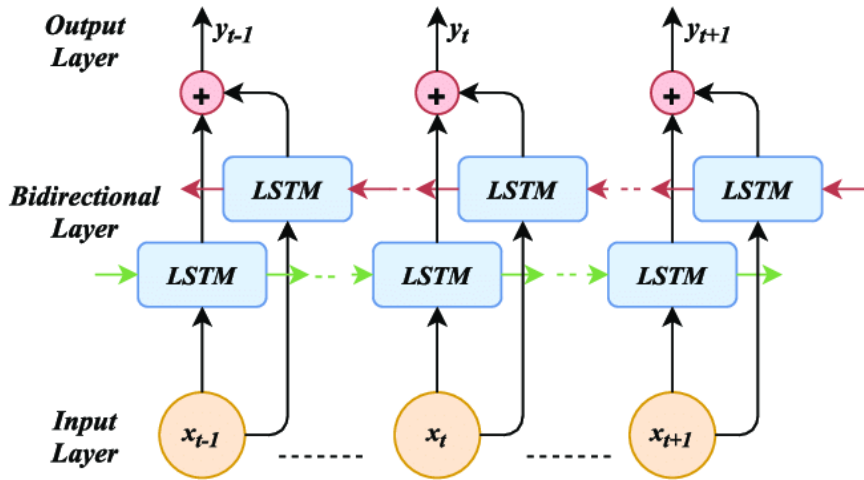


Figure 3.15: *Bidirectional LSTM model showing the input and output layers. The red arrows represent the backward sequence track and the green is the forward sequence track.*

3.2.4 Multi-Head Self-Attention (MSA)

Introduced in [25]. MSA-based networks have shown to be very effective in NLP tasks, such as machine translation, language modeling, and text classification.

3.2.4.1 Attention

The attention function can be understood as a method that takes a query and a collection of key-value pairs and produces an output. All of these elements - the query, keys, values,

and output - are vector forms. The output is determined by a weighted sum of the values. Each value's weight is calculated using a compatibility function that matches the query with its associated key.

We can break down the concept of an attention function into:

- **Query, Keys, Values:** These are all vectors. In the context of a transformer model, for example, these vectors are created by multiplying the input vectors (like word embeddings in NLP) with weight matrices that the model learns during training.
- **Mapping:** The attention function maps a query and a set of key-value pairs to an output. This is done through a series of mathematical operations.
- **Compatibility Function:** This is a function that calculates the compatibility or similarity between the query and each key. The result of this function is used to weight the corresponding value. In many models, the dot product is used as the compatibility function.
- **Weighted Sum of Values:** After each value is weighted according to the compatibility of its corresponding key with the query, these weighted values are summed to produce the output. This output is a vector that represents the input elements but with attention applied - elements that are deemed more important (have higher compatibility with the query) have a greater presence in the output.

This mechanism allows a model to focus on different parts of the input when producing each part of the output, hence the term "attention". It's a crucial part of many modern neural network architectures, especially in the field of natural language processing.

Attention mechanisms have also been adopted in audio signal processing, particularly in tasks like speech recognition and sound classification. In these tasks, attention mechanisms can help the model focus on certain parts of the audio signal that are more relevant to the task at hand. For instance, in speech recognition, an attention mechanism can allow the model to focus more on the parts of the audio signal where the speech is present and less on the background noise.

3.2.4.2 Scaled Dot-Product Attention

Scaled dot-product attention is a type of attention function that has been used in many NLP models, including the original transformer model. It takes a query and a set of key-value pairs and produces an output. The query, keys, values, and output are all vectors. The output is calculated as a weighted sum of the values, where the weight assigned to each value is calculated using a compatibility function that matches the query with its associated key.

The compatibility function used in scaled dot-product attention is the dot product. The dot product is calculated between the query and each key, then the result is divided by the square root of the dimension of the query vector. This scaling factor is used to prevent the dot product from growing too large, which can cause problems with the softmax function that is used to calculate the weights. The softmax function is a function that takes a vector as input and produces a vector as output, where each element of the output vector is in the range $[0, 1]$ and the sum of the elements is 1. The softmax function is used to

calculate the weights assigned to each value in the weighted sum of values that is the output of the attention function.

The scaled dot-product attention function can be expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.5)$$

where Q is the query vector, K is the matrix of key vectors, V is the matrix of value vectors, and d_k is the dimension of the query vector.

3.2.4.3 Multi-Head Attention

Instead of using a single attention function with keys, values, and queries all having a dimension of d_{model} , it's found to be advantageous to project the queries, keys, and values h times linearly with different learned linear projections to dimensions d_k , d_k , and d_v respectively. The attention function is then performed in parallel on each of these projected versions of queries, keys, and values, producing d_v -dimensional output values. These outputs are then concatenated and projected again to yield the final values.

This approach, known as multi-head attention, allows the model to attend to information from different representation subspaces at different positions simultaneously. This is something that can't be achieved with a single attention head due to averaging.

Multi-head attention can be expressed as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3.6)$$

Where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Here, the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

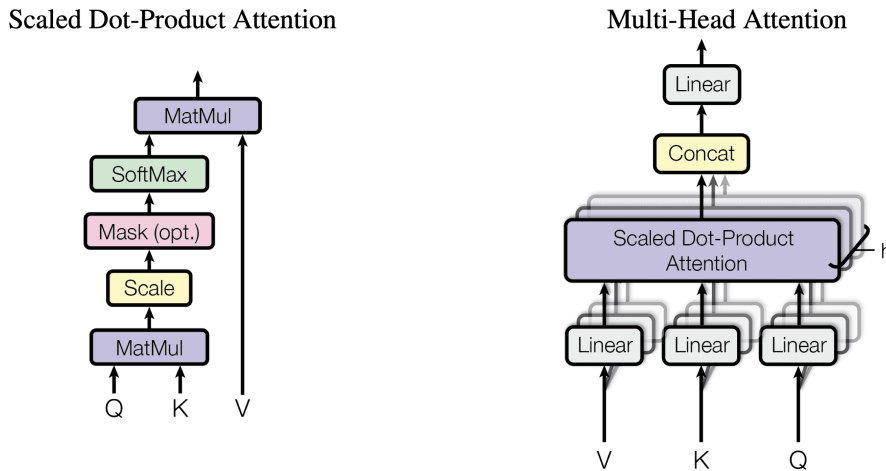


Figure 3.16: (left) *Scaled Dot-Product Attention*. (right) *Multi-Head Attention consists of several attention layers running in parallel.*

3.2.5 UNet Architectures

Introduced in [26], U-Net is a type of convolutional neural network that's often used for biomedical image segmentation. It's named U-Net because of its U-shaped architecture.

The U-Net architecture consists of two main parts: the encoder (also known as the contracting or downsampling path) and the decoder (also known as the expanding or upsampling path).

3.2.5.1 Encoder

The encoder part of U-Net is responsible for capturing the context in the image. It works by applying repeated steps of convolution, each followed by a rectified linear unit (ReLU) and a max pooling operation. These steps help to reduce the spatial dimension of the input while increasing the depth, thereby capturing the context in the image. The output of the encoder is a set of feature maps that represent the high-level features of the input image.

3.2.5.2 Decoder

The decoder part of U-Net is responsible for localizing and segmenting the object of interest in the image. It works by applying a series of up-convolutions and concatenations with the correspondingly cropped feature map from the encoder path, followed by regular convolutions. These steps help to increase the spatial dimension and decrease the depth of the feature maps, thereby reconstructing the segmented image from the high-level features. The output of the decoder is the segmented image.

3.2.5.3 Skip Connections

The key feature of U-Net is the skip connections between the encoder and decoder. These connections allow the decoder to use information from the encoder at the same level, which helps to preserve the details and localize the object of interest in the image.

U-Net can be used for audio enhancement tasks in both the time and frequency domains. The general approach is similar in both cases: the U-Net learns to map from noisy signals to clean signals. However, the specifics of how the input and output are represented, and how the U-Net is trained, can vary depending on whether you're working in the time domain or the frequency domain.

3.2.5.4 Speech Enhancement with U-Net

In the time domain, the input to the U-Net would be a noisy audio signal, and the target would be the clean audio signal. During training, the U-Net learns to map from the noisy signal to the clean signal by minimizing a loss function that measures the difference between the U-Net's output and the clean signal. After training, you can use the U-Net to enhance new audio signals by inputting a noisy signal and getting out an enhanced signal.

In the frequency domain, the input to the U-Net would be a spectrogram of a noisy audio signal, and the target would be the spectrogram of the clean audio signal. A spectrogram is a 2D representation of an audio signal that shows how the frequencies present in

the signal change over time. During training, the U-Net learns to map from the noisy spectrogram to the clean spectrogram by minimizing a loss function that measures the difference between the U-Net's output and the clean spectrogram. After training, you can use the U-Net to enhance new audio signals by inputting a spectrogram of a noisy signal and getting out a spectrogram of an enhanced signal. The output spectrogram can then be transformed back into the time domain to get the enhanced audio signal.

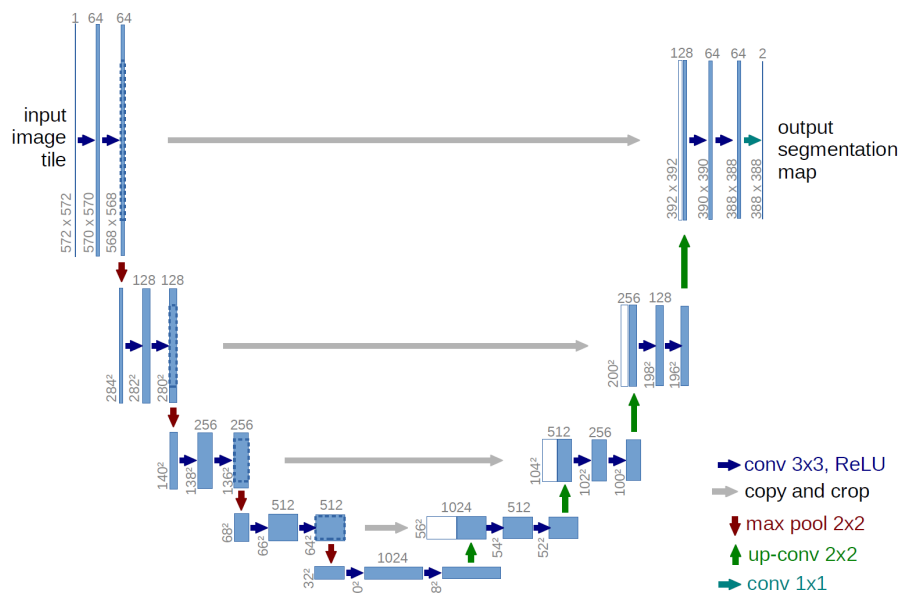


Figure 3.17: *U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multichannel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations [26].*

Chapter 4

Datasets

4.1 Valentini [27]

Also known as the Noisy speech database for training speech enhancement algorithms and TTS models.

4.1.1 Training Dataset

The training dataset consists of clean and noisy parallel speech audio tracks. It was designed for training and testing speech enhancement methods operating at 48kHz. It consists of 2 datasets. They are described as below:

1. 28 Speakers dataset: 14 males and 14 females from the same region and accent, which was England.
2. 56 Speakers dataset: 28 males and 28 females from 2 regions, which were the United States and Scotland.

Valentini dataset contained around 400 sentences per speaker, and it used 10 different noise types:

1. Speech-shaped noise, which was created by filtering white noise through a frequency response filter matching the speech level of a male speaker.
2. Babble noise, which was created by adding selected speech from 6 speakers who were not used in training or testing data.
3. A real home domestic noise from inside a kitchen.
4. A real office noise from a meeting room.
5. A real public space noise from a cafeteria.
6. A real public space noise from a restaurant.
7. A real public space noise from a subway station.
8. Transportation noise from a car.
9. Transportation noise from a metro.

10. Street noise from a busy traffic intersection.

The real noises were chosen from the DEMAND dataset [28]. Four different signal-to-noise ratios (SNR) were chosen: 15 dB, 10 dB, 5 dm, and 0 dB. Which resulted in 40 different noise conditions. There were around 10 different sentences in each condition, resulting in 400 sentences when mixed with noise. After normalizing the clean waveforms, they were mixed in with the noise, and silent segments longer than 200 ms were removed from the beginning and the end of each sentence.

4.1.2 Testing Dataset

For the testing dataset, 2 other voices from England were chosen. New 5 noises were chosen: a living room noise, an office space noise, a bus noise, an open cafeteria noise, and a public square noise. Slightly higher SNR values were used: 17.5 dB, 12.5 dB, 7.5 dB, and 2.5 dB. This resulted in 20 different noise conditions which were added to the clean testing dataset as described before.

4.2 DNS [1], [29], [30]

The Deep Noise Suppression Challenge is a yearly challenge organized by Microsoft Research. The first challenge was introduced in 2020, and it has been held annually since. It focuses especially on Speech Enhancement tasks and real-time settings. The Deep Noise Suppression (DNS) challenge aims to unify research activity in the Speech Enhancement domain by making the train/test datasets and subjective assessment system available to the public. It releases large clean speech and noise datasets. It also provides configurable synthetization scripts for producing the training datasets. It also releases a part of the test dataset.

4.2.1 DNS 2020 [29]

4.2.1.1 Clean Dataset

The clean speech dataset is obtained from Librivox, which is a public audiobook dataset. It has the audiobook recordings of over 10000 public domain books in multiple languages, the majority of which are in English [31]. Although most of these recordings were of excellent speech conditions and quality, they had many recordings with poor conditions such as speech distortion, background noise, and reverberation. These poor-conditioned recordings needed to be removed, as the clean dataset must be free of noisy and poor-conditioned recordings. An online subjective test framework ITU-T P.808 was used to keep only excellent quality recordings. Mean Opinion Score (MOS) was used as a metric by keeping only the upper quartile with respect to MOS. The final clean dataset included 500 hours of speech from 2150 speakers. The filtered clips are then divided into 10-second segments.

4.2.1.2 Noise Dataset

The noise dataset was selected from Audioset [32] and Freesound. Audioset consists of 2 million sound clips of 10s each, where each sound clip is human-labelled. These clips are extracted from YouTube videos belonging to about 600 audio events. Some classes are

overrepresented compared to others. A sampling approach was executed to balance each class by having at least 500 clips. A speech activity detector was used to remove any clips containing speech activity. The resulting dataset contained about 150 classes and 60000 clips. An additional 10000 noise clips related to VoIP applications were downloaded from the Freesound and DEMAND dataset [28].

4.2.1.3 Noisy Dataset

The noisy dataset is the addition of clean speech from the clean dataset and a noise clip from the noise dataset at various SNR levels. The SNR is only computed using active segments of both the clean and noise signals. This was done to avoid the overshoot of amplitude in impulsive noise (i.e., door shutting or dog barking).

4.2.2 DNS 2022 [30]

This Deep Noise Suppression (DNS) Challenge was the 4th DNS challenge. The original authors open-sourced the datasets and test sets for researchers to train their deep noise suppression models as well as a subject evaluation framework based on ITU-Y P.835 to rate and rank-order the challenge entries. In this challenge specifically, a new change was introduced: (i) included mobile devices scenarios in the blind test set; (ii) Included a personalized noise suppression track with baseline; (iii) Added WAcc as an objective metric; (iv) Included DNSMOS P.835; (v) Made the training datasets and test sets full band (48 kHz).

4.2.2.1 Clean Dataset

The clean speech dataset consists of six languages, namely English, French, German, Italian, Russian, and Spanish. English clean speech consists of read speech and singing, while the rest of the languages only have read speech. The original authors combined all audio clips from each unique talker into a single file. The PDNS training data has a total of 3,230 talkers, out of which 60% of the talkers are randomly chosen to be the primary talkers while the rest are neighboring talkers. They also provided the file list for PDNS clean speech with 'primary' and 'secondary' tags, sampling 60% of the speakers as primary talkers, ensuring a uniform distribution among all languages, read speech or singing. English speech was derived from Librivox like previous versions mentioned above. English singing data consists of high-quality audio recordings from professional singers contained in the VocalSet corpus. It has 10.1 hours of clean singing from 20 singers: 9 males and 11 females. The PDNS English clean speech contains 1934 talkers from Librivox, 110 talkers from VCTK corpus, and 20 talkers from Vocalset. The PDNS clean speech consists of 47 talkers for French, 874 for German, 14 for Italian, 7 for Russian, and 224 for Spanish.

4.2.2.2 Noise Dataset

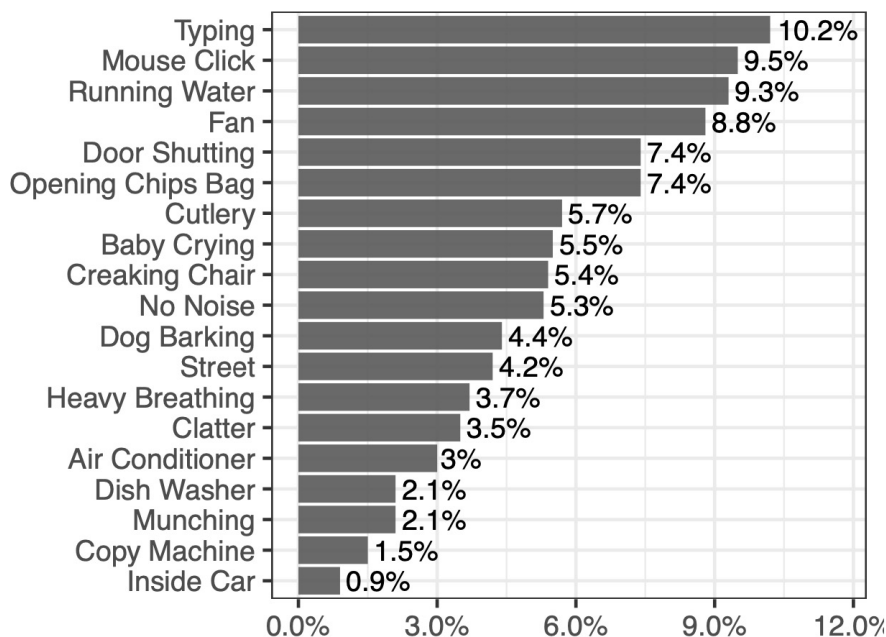


Figure 4.1: *Distribution of noise types in DNS 2022 blind test set.*

Noise data consists of about 62,000 clips belonging to 150 noise classes. Audio Set is a collection of about 2 million human-labeled 10s sound clips drawn from YouTube videos belonging to about 600 audio events. Approximately 42% of the clips have a single class, but the rest may have 2 to 15 labels. The authors used a speech activity detector to remove the clips with any kind of speech activity (voice content) from our noise clips. The total noise data constitute 181 hours of audio. Fig. 4.1 shows the histogram of noise classes included in the blind test set. These noise types were validated by a human listener after the collection of the blind set

4.2.2.3 Impulse Response

They offered over 60,000 synthetic and 248 real room impulse responses, which can be used to create training data with reverberant noise. Depending on the setup selected, the training data synthesizer adds noise to reverberant clean speech. For their DNS models, participants can select between training clean speech and reverberant speech.

4.2.3 DNS 2023 [1]

The fifth DNS challenge’s goal is to create models for the cooperative detection, suppression, and dereverberation of disruptive talkers. Certain aspects of the primary talker’s speech, like direct-to-reverberation (DRR) and signal-to-noise ratio (SNR), allow for the suppression of nearby talkers when they wear headphones—even in the absence of enrollment information for the primary talker. The authors were inspired by this to produce two tunes for this challenge: Track-2 Speakerphone; and Track-1 Headset. Each test clip has corresponding enrollment data (10–30 seconds duration) for the principal talker, and

both tracks have full band (48kHz) training data and test sets. Both personalized and non-personalized model submissions were accepted for each track, and all submissions were submitted to the same subjective evaluation. The majority of models that were entered into the challenge were customized models. The winning team in both tracks had the best models with improvements in the challenge’s score of 0.145 and 0.141 when compared to the noisy blind test set.

4.2.3.1 Challenge Dataset

To extract the near-field headset-like clean speech, which is released as clean speech for the headset track, they employed a machine learning model. For the speakerphone track, the authors utilized the full clean voice from the fourth DNS Challenge. The 4th DNS Challenge’s noise dataset and impulse responses are being used here. By convolving near-field clean speech from Track 1 with impulse responses, participants can repurpose it to generate training data for Track 2. The offered datasets come in a number of talker and device varieties and languages. Additionally, they furnished speaker ID data for every uncontaminated speech clip to expedite the creation of customized models for both tracks. Moreover, they also offered code for extracting speaker embeddings from cutting-edge ECAPA-TDNN embeddings that have been trained on Voxceleb. The authors supplemented training data with clean speech that included emotions like laughing, sobbing, yelling, or singing. Clean speech clips in ten non-English languages and English make up the entire training package. A total of 760.53 hours of clean speech—read speech (562.72 hours), singing voice (8.80 hours), emotional speech (3.6 hours), and Chinese Mandarin (185.41 hours)—are included in the Track-2 training set. By extracting near-end speech from the Track-2 dataset, clean speech from Track-1 is achieved. The four subsets of clean speech are as follows: (i) Read speech recorded in a clean environment; (ii) Singing clean speech; (iii) Emotional clean speech; and (iv) clean speech that is not in English.

4.3 LibriSpeech

LibriSpeech [33] is a corpus of approximately 1000 hours of 16kHz read English speech, prepared by Vassil Panayotov with the assistance of Daniel Povey. The data is derived from reading audiobooks from the LibriVox project and has been carefully segmented and aligned.

The majority of the recordings are based on publicly available texts from Project Gutenberg. While the use of audiobooks to create synthetic voices has been studied in the past, as far as we know, there isn’t a read speech corpus as large as the one we present here that is freely accessible in English and sufficient for training and testing speech recognition systems.

The read speech data set LibriSpeech, which is derived from LibriVox’s audiobooks, is presented in this study. The highly permissive CC BY 4.0 license makes the corpus freely available, and the open-source Kaldi ASR toolbox has example scripts that show how well-performing acoustic models can be trained using this data.

The dataset is mainly divided into 3 sets with each of them being divided into a few different versions as shown in Table 4.1.

Subset	Hours	Per-speaker Minutes	Female Speakers	Male Speakers	Total Speakers
dev-clean	5.4	8	20	20	40
test-clean	5.4	8	20	20	40
dev-other	5.3	10	16	17	33
test-other	5.1	10	17	16	33
train-clean-100	100.6	25	125	126	251
train-clean-360	363.6	25	439	482	921
train-other-500	496.7	30	564	602	1166

Table 4.1: *Data subsets in LibriSpeech*

4.3.1 Training Set

There are 3 versions of the training set:

- **train-clean-100:** 100 hours of “clean” speech
- **train-clean-360:** 360 hours of “clean” speech
- **train-other-500:** 500 hours of “other” speech

Where “clean” resembles clean speech audio and “other” can have clean speech and also not-so-clear speech and might have noisier samples.

4.3.2 Development Set

There are 2 versions of the development set:

- **dev-clean:** 5 hours of “clean” speech
- **dev-other:** 5 hours of “other”, more challenging, speech

Where “clean” resembles clean speech audio and “other” can have clean speech and also not-so-clear speech and might have noisier samples.

4.3.3 Test Set

There are 2 versions of the test set:

- **test-clean:** test set, “clean” speech
- **test-other:** test set, “other” speech

Where “clean” resembles clean speech audio and “other” can have clean speech and also not-so-clear speech and might have noisier samples.

4.4 Collected Dataset

4.4.1 Motivation

The personalized DNS dataset was extensively gathered, but its training samples did not accurately represent the problem we aimed to solve. The DNS dataset prioritized

quantity over quality, a common strategy in deep learning for rapid and cost-effective data collection. Due to limited computing resources, we initially trained our models using only 100 hours of data from the 500 hours available in the personalized DNS dataset. Unfortunately, the results of these experiments were unsatisfactory, as the dataset lacked both quality and quantity.

4.4.2 Data Collection

The dataset replicates the scenario of two speakers, Speaker A and Speaker B, speaking simultaneously and provides a separate clean reference speech for Speaker A.

4.4.2.1 Train-validation split

The train-validation split, consisting of 30 speakers with 10 clean 10-second utterances each. Among the speakers, 22 were male and 8 were female, all speaking Arabic. For each speaker, pairs of his utterances were generated, pairs with identical utterances were replaced with silence as the clean speech. To each pair, 10 samples from other speakers were added for speaker B. The total number of samples in the train-validation split is 30000. The validation set represents 1% of the train-validation split, totaling 300 samples.

4.4.2.2 Test split

The test split was used to assess the model's performance with new speakers. It comprises 6 speakers, each providing 5 clear 10-second utterances. These individuals include 2 male Arabic speakers, 3 female Arabic speakers, and 1 female English speaker. This distribution was deliberately chosen to gauge the model's ability to generalize to different speakers. The generation process is akin to the train-validation split, except the inclusion of silence samples. The test split consists of a total of 1200 samples.

Chapter 5

Evaluation Metrics

The ability to accurately assess speech quality and intelligibility is critical for diverse applications, from telecommunications to voice assistants. To objectively evaluate these aspects, a range of metrics have been developed, classified into **objective** and **subjective** categories. **Objective metrics** rely on well-defined algorithms and calculations, while **subjective metrics** draw upon human perception through listening tests or related techniques. Additionally, some metrics require a reference signal for comparison (**intrusive**), while others operate solely on the processed audio (**non-intrusive**). Understanding these distinctions is crucial for selecting the appropriate metric for specific evaluation needs.

5.1 Objective Metrics

5.1.1 Signal-to-Noise Ratio (SNR)

Signal-to-Noise Ratio (SNR) is a measure of the ratio of the power of a signal to the power of background noise. It is defined as:

$$SNR = 10 \log_{10} \frac{P_{signal}}{P_{noise}} \quad (5.1)$$

Where P_{signal} is the power of the signal and P_{noise} is the power of the noise. SNR is usually measured in decibels (dB). The higher the SNR, the better the quality of the signal. SNR is a common metric used in speech enhancement tasks. It is used to measure the quality of the enhanced speech signal by comparing it to the original clean speech signal. The higher the SNR, the better the quality of the enhanced speech signal.

5.1.2 Perceptual Evaluation of Speech Quality (PESQ)

Perceptual Evaluation of Speech Quality (PESQ) is a method for assessing the quality of speech signals. It is a standardized method for evaluating the quality of speech signals. It is widely used to assess narrow-band telephone network and speech codec quality. PESQ compares processed speech to the original, focusing on characteristics like sharpness, noise, clipping, and volume. Scores range from -0.5 to 4.5, with higher values indicating better quality. The calculation steps are as follows:

- Reference audio and processed speech signals are time-aligned.
- A perceptual model is employed to translate signals into an auditory representation. This model relies on power distributions across the time-frequency domain, it relies on compressive loudness scaling too.
- Differences between signals and their mappings are considered, where positive variances suggest the presence of components like noise, while negative variances suggest the omission of components.

This metric offers a limited assessment of transmission quality, focusing only on one-way speech distortion and noise in speech quality. So, factors such as loudness loss, delay, sidetone, and echo are not accounted for in the PESQ scores.

As we will see in the paper's results, PESQ scores are sometimes represented as WB, and sometimes as NB. WB stands for wide bandwidth, while NB represents narrow bandwidth. While the initial PESQ algorithm was designed for narrow-band speech (8 kHz sampling rate), the experiments utilized extensions for wideband speech (16 kHz).

5.1.3 Short-Time Objective Intelligibility (STOI)

Short-Time Objective Intelligibility (STOI) is a method for assessing the intelligibility of speech signals. It predicts the intelligibility of noisy speech, not overall quality. STOI exhibits a strong correlation with intelligibility in various distorted speech scenarios such as those affected by additive noise, single-channel noise reduction, multi-channel noise reduction, binary masking, and vocoded speech.

It is calculated by taking the mean of Pearson's correlation coefficient between the clean and the distorted envelopes for each sort-time frame and frequency band. The steps are as follows:

- Signals are decomposed by a 1/3-octave filter band and then segmented into short-time windows.
- They are then normalized, to compensate for different playback levels.
- They are then clipped, to set an upper bound on the severity of degradation a unit of speech time frequency can be.
- Finally, signals are compared using the correlation coefficient as stated above.

Scores range from 0 to 1 (typically represented as %) and represent predicted intelligibility.

5.1.4 Mean Opinion Score (MOS)

Available in both objective and subjective forms. Subjective MOS reflects the average rating from human listeners in quality evaluation tests. Objective MOS is also referred to as predicted MOS.

MOS can be used to assess the comprehensive voice quality in calls and video sessions, it can be utilized for various audio codecs (e.g., MP3, Vorbis, AAC, Opus), video codecs (e.g., H.264, VP8), and in streaming sessions where network effects may impact communication quality negatively.

It cares for characteristics such as bandwidth, latency, packet loss, and compression ratio in the codec version. Its score ranges from 1 to 5, where 1 means the worst score and 5 is the highest. Below 3.1 represents poor call quality, while above 4.34 represents the best call quality range.

MOS is calculated by taking the arithmetic mean over single ratings. The most objective MOS context we care for is DNSMOS, which stands for ‘deep noise suppression MOS’, while the subjective MOS we care for depends on predictions concerning that same context.

The DNSMOS computes 3 scores, each on a scale from 1 to 5, which are:

- SIG (Signal, speech quality).
- BAK (background noise quality).
- OVRL (overall quality).

5.1.5 DNSMOS P.808 [34]

Layer	Output dimension
Input	900 x 120 x 1
Conv: 32, (3 x 3), ‘ReLU’	900 x 120 x 32
MaxPool: (2 x 2), Dropout(0.3)	450 x 60 x 32
Conv: 32, (3 x 3), ‘ReLU’	450 x 60 x 32
MaxPool: (2 x 2), Dropout(0.3)	225 x 30 x 32
Conv: 32, (3 x 3), ‘ReLU’	225 x 30 x 32
MaxPool: (2 x 2), Dropout(0.3)	112 x 15 x 32
Conv: 64, (3 x 3), ‘ReLU’	112 x 15 x 64
GlobalMaxPool	1 x 64
Dense: 64, ‘ReLU’	1 x 64
Dense: 64, ‘ReLU’	1 x 64
Dense: 1	1 x 1

Table 5.1: *Architecture of the DNSMOS Prediction P.808 model.*

The most reliable approach to evaluating Speech Enhancement (SE) techniques is subjective speech quality evaluation. Subjective tests, however, are not scalable since they need a large number of listeners and are expensive, time-consuming, and labor-intensive to conduct. Speech Enhancement (SE) algorithms designed for human perception are commonly evaluated using traditional objective speech quality measures including Signal to Distortion Ratio (SDR), Perceptual Evaluation of Speech Quality (PESQ), and Perceptual Objective Listening Quality Analysis (POLQA). To calculate the quality score, there are additional intrusive measurements that need a reference clean speech. However, when applied to SE tasks involving perceptually invariant alterations, they are demonstrated to have a poor correlation with human evaluation. Also, intrusive metrics cannot be used to evaluate real recordings when clean reference is unavailable in realistic scenarios.

The authors developed DNSMOS, a reliable objective perceptual speech quality metric. DNSMOS got the name because it can be used to accurately stack rank various Deep Noise Suppression (DNS) techniques based on MOS calculations. Human ratings collected

using ITU-T P.808 are used to train a basic Convolutional Neural Network (CNN) based model. P.808 is a highly repeatable online subjective testing methodology that has been demonstrated to accurately stack rank noise suppression models when tested as an average over a statistically significant number of clips.

The 600 noisy speech test clips that the authors used were a mix of actual and artificial recordings. There are both reverberant and non-reverberant clips in the synthetic test set. The actual recordings were made at various goal levels, Signal to Noise Ratios (SNR), and noise kinds. There are more than 100 different noise kinds and speakers in the test set. Over 200 distinct noise reduction techniques were applied to these 600 clips. Some of these methods improved the overall speech quality, but they also introduced a variety of artifacts as a consequence of over or under-suppression of noise, resulting in speech and noise distortions. The speech quality ratings of the processed clips varied from very poor (MOS=1) to excellent (MOS=5). In total, the authors used about 120,000 audio clips with associated MOS scores as ground truth. The average length of each audio clip was about 9 seconds.

Log power Mel spectrogram was used as an input feature as it correlates well with human perception and is proven to work very well for analyzing speech quality. For spectral features, a frame size of 20 ms with a hop length of 10 ms and 120 Mel frequency bands were used. The input features are then converted to a dB scale. The authors explored different configurations of CNN-based models. The architecture for the best-performing model is shown in Table 5.1.

5.1.6 DNSMOS P.835 [35]

Human subjective evaluation is considered the "gold standard" for assessing speech quality as it is optimized for human perception. Perceptual objective metrics are used as proxies for these subjective scores. In this context, the authors introduced a new non-intrusive speech quality metric named Deep Noise Suppression Mean Opinion Score (DNSMOS). This metric is based on the ITU-T Rec. P.835 subjective evaluation framework, providing separate quality scores for speech, background noise, and overall quality.

The authors trained a deep learning model to predict P.835 human ratings, resulting in three distinct scores: Signal (SIG), Background (BAK), and Overall (OVRL) quality. The model demonstrated a high correlation with human ratings, achieving a Pearson's Correlation Coefficient (PCC) of 0.94 for SIG, and 0.98 for both BAK and OVRL.

This work was an extension to the previous metric DNSMOS P.808 [34], which was designed to predict the overall quality of speech signals. The new metric, DNSMOS P.835, is more comprehensive as it predicts the quality of both speech and background noise separately in addition to the overall quality. The authors used the subjective labels obtained from ITU-T P.835 from Deep Noise Suppression (DNS) Challenge 3 [36] and the noisy clips processed by several noise suppression models internally at Microsoft. In total 30,000 audio clips with associated MOS scores were used as ground truth. The average length of each audio clip was about 9 seconds, resulting in a total of 75 hours of audio data.

Log power spectrogram was used as an input feature extracted over 9 seconds duration as it correlates well with human perception and is proven to work very well for analyzing

Layer	Output dimension
Input	900 x 120 x 1
Conv: 128, (3 x 3), 'ReLU'	900 x 161 x 128
Conv: 64, (3 x 3), 'ReLU'	900 x 161 x 64
Conv: 64, (3 x 3), 'ReLU'	900 x 161 x 64
Conv: 32, (3 x 3), 'ReLU'	900 x 161 x 32
MaxPool: (2 x 2), Dropout(0.3)	450 x 80 x 32
Conv: 32, (3 x 3), 'ReLU'	450 x 80 x 32
MaxPool: (2 x 2), Dropout(0.3)	225 x 40 x 32
Conv: 32, (3 x 3), 'ReLU'	112 x 20 x 32
MaxPool: (2 x 2), Dropout(0.3)	112 x 15 x 32
Conv: 64, (3 x 3), 'ReLU'	112 x 20 x 64
GlobalMaxPool	1 x 64
Dense: 128, 'ReLU'	1 x 128
Dense: 64, 'ReLU'	1 x 64
Dense: 1 or 3	1 x 1 or 1 x 3

Table 5.2: *Architecture of the DNSMOS Prediction P.835 model.*

speech quality. For spectral features, they used a frame size of 20 ms with a Hamming window and a hop length of 10 ms. The input features are then converted to a dB scale.

In the training phase, the authors trained two different models with almost the same architecture except for the last layer. One model is trained to predict all 3 outputs (SIG, BAK, OVRL) and the other model is trained to predict only SIG. It was found that the prediction of SIG is a much harder task and is less correlated with BAK and OVRL. The models were trained with a batch size of 32 using the Adam optimizer [37] and MSE loss function until the loss saturated. The authors explored different network architectures and the model in Table 5.2 generalized the best and was of the least complexity.

5.1.7 Real-time Factor (RTF)

Real-time Factor (RTF) is a measure of the ratio of the time required to process a signal to the duration of the signal. It is defined as:

$$\text{RTF} = \frac{\text{Processing time}}{\text{Duration of signal}} \quad (5.2)$$

RTF compares simulated processing time to real-time, with values above 1 indicating faster than real-time processing and lower RTF favors real-time.

5.1.8 Word Error Rate (WER)

It assesses the accuracy of speech-to-text transcription by calculating the ratio of errors (substitutions, insertions, deletions) to the total number of spoken words.

Transcription errors could be substitutions, insertions, or deletions. And since WER is defined as the ratio between the number of errors in a transcript to the total words spoken, then it can be calculated as follows:

$$\text{Word Error Rate} = \frac{\text{Substitutions} + \text{Insertions} + \text{Deletions}}{\text{Number of Words Spoken}} \quad (5.3)$$

A lower WER indicates better accuracy, with state-of-the-art systems achieving error rates close to human transcriptionists (around 4%).

5.2 Subjective Metrics

5.2.1 Subjective Mean Opinion Score (SMOS)

Subjective MOS is a metric used to assess the perceived quality of audio in telecommunications and related fields. It differs from objective MOS, which relies on algorithms and calculations, by relying on the judgment of human listeners. This human evaluation provides insight into how users experience the quality of the media, which can be crucial in optimizing systems and algorithms for real-world applications.

Subjective MOS is evaluated through listening tests. These tests can vary in format but often involve a group of listeners rating the quality of audio or video samples on a predefined scale (e.g., from 1 to 5). The test environment needs to be controlled to minimize external influences, ensuring consistency and accurate results.

Subjective MOS collects three scores:

- CSIG (Mean opinion score predictor of signal distortion).
- CBAK (MOS predictor of background-noise intrusiveness).
- COVL (MOS predictor of overall signal quality).

Chapter 6

Related Work

This chapter discusses the current state of the art in speech enhancement, and how it relates to the problem definition.

6.1 DEMUCS

Défossez et al. [3] proposed a real-time causal speech enhancement model that operates directly on raw waveforms. The authors adopted an encoder-decoder architecture with skip connections. This model, which has been optimized across time and frequency domains using several loss functions, has been empirically shown to be effective at eliminating a wide range of background noises, including both stationary and non-stationary forms, as well as room reverb. Evaluations using common benchmarks, which include objective metrics and human judgments, show that the model achieves cutting-edge performance for both causal and non-causal approaches while operating directly on the raw waveform.

Inspired by recent improvements, the authors suggest a real-time adaption of the DEMUCS [38] architecture for speech enhancement. This architecture uses a causal model based on convolutions and LSTMs, with a frame size of 40ms and a stride of 16ms, and executes faster than real-time on a single laptop processor core. To improve audio quality, the model uses hierarchical generation (similar to U-Net[26] skip-connections) to transition from waveform to waveform.

6.1.1 Model Architecture

DEMUCS consists of a multi-layer convolutional encoder and decoder with U-net [26] skip connections, and a sequence modeling network applied on the encoders' output. It is characterized by its number of layers L , the initial number of hidden channels H , layer kernel size K and stride S , and resampling factor U . The encoder and decoder layers are numbered from 1 to L (in reverse order for the decoder, so layers at the same scale have the same index). As the main focus is on monophonic speech enhancement, the input and output of the model have a single channel only.

Formally, the encoder network E receives the raw waveform as input and produces a latent representation $E(x) = z$. Each layer consists of a convolution layer with a kernel

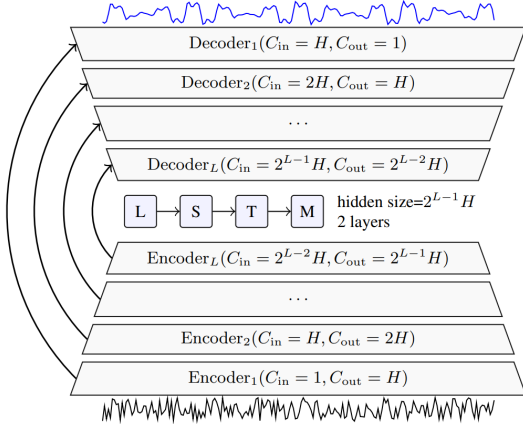


Figure 6.1: *Causal DEMUCS with the noisy speech as input on the bottom and the clean speech as output on the top. Arrows represent U-Net skip connections. H controls the number of channels in the model and L its depth.*

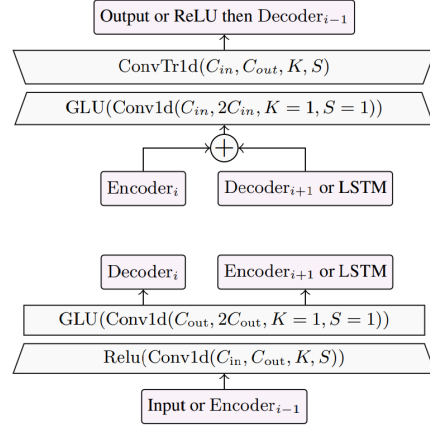


Figure 6.2: *View of each encoder (bottom) and decoder layer (top). Arrows are connections to other parts of the model. C_{in} (resp. C_{out}) is the number of input channels (resp. output), K the kernel size, and S the stride.*

size of K and stride of S with $2^{i-1} H$ output channels, followed by a ReLU activation, a 1×1 convolution with $2^i H$ output channels, and finally a GLU activation that converts the number of channels back to $2^{i-1} H$ as demonstrated in Figure 6.2.

A sequence modeling R network uses the latent representation z as input and produces a non-linear transformation of the same size, $R(z) = LSTM(z) + z$, denoted as $\hat{z}$. The LSTM network is composed of two layers and $2^{L-1} H$ hidden units. For causal prediction, a unidirectional LSTM is employed, whereas for non-causal models, a bidirectional LSTM is used, followed by a linear layer that combines the two outputs.

Finally, a decoder network D accepts $\hat{z}$ as input and returns an estimated clean signal $D(\hat{z}) = \hat{y}$. The i -th layer of the decoder takes as input $2^{i-1} H$ channels and applies a 1×1 convolution with $2^i H$ channels, followed by a GLU activation function that outputs $2^{i-1} H$ channels and finally a transposed convolution with a kernel size of 8, stride of 4, and $2^{i-2} H$ output channels, accompanied by a ReLU function. For the last layer, the output is a single channel with no ReLU. As shown in Figure 6.1, a skip connection connects the encoder's i -th layer output to the decoder's i -th layer input.

6.1.2 Experimental Setup

6.1.2.1 Dataset

The authors used a synthesized subset of the DNS 2020 dataset [29] and Valentini [27] dataset. They also applied several augmentation techniques:

- **Random Shift:** randomly shifts the signal in time.
- **RevEcho:** adds decaying echoes to the training data.
- **Remix:** shuffles noises within a local batch to form new noisy mixtures.
- **BandMask:** removes 20% of frequencies uniformly at mel scale.

The random shift was applied to all datasets, Remix and BandMask was applied to Valentini [27], and RevEcho was applied to DNS [29].

6.1.2.2 Training

The authors trained the DEMUCS model for 400 epochs on the Valentini [27] dataset and 250 on the DNS [29] dataset. They used the L1 loss between the predicted and ground truth clean speech waveforms, and for the Valentini dataset, also added the STFT loss, which is defined as the sum of the *spectral convergence* (*sc*) loss and the *magnitude* (*mag*) loss as follows:

$$\begin{aligned}
 L_{\text{stft}}(y, \hat{y}) &= L_{\text{sc}}(y, \hat{y}) + L_{\text{mag}}(y, \hat{y}) \\
 L_{\text{sc}}(y, \hat{y}) &= \frac{\| |STFT(y)| - |STFT(\hat{y})| \|_F}{\| |STFT(y)| \|_F} \\
 L_{\text{mag}}(y, \hat{y}) &= \frac{1}{T} \|\log |STFT(y)| - \log |STFT(\hat{y})|\|_1
 \end{aligned} \tag{6.1}$$

where y and $\hat{y}$ are the clean signal and the enhanced signal respectively, $\|\cdot\|_F$ and $\|\cdot\|_1$ are the Frobenius and L_1 norms respectively. The STFT loss was added with a weight of 0.5. They used the Adam optimizer with a step size of $3e^{-4}$, a momentum of $\beta_1 = 0.9$ and a denominator momentum $\beta_2 = 0.999$. For the Valentini dataset, the authors used the original validation set and kept the best model, for the DNS dataset they trained without a validation set and kept the last model. The audio is sampled at 16 kHz.

6.1.3 Conclusion

The paper presents a novel causal speech enhancement model that operates in the raw waveform domain and runs in real-time on a laptop CPU. The model is capable of removing various types of background noise and has been optimized for both time and frequency domains. The proposed model also includes data augmentation techniques to further improve its performance and generalization abilities. Evaluations on standard benchmarks show that the proposed model matches the state-of-the-art performance of both causal and non-causal methods while working directly on the raw waveform. The model is also found to be beneficial for improving an ASR model under noisy conditions. Overall, the proposed model presents a promising solution for real-time speech enhancement in various applications.

6.2 CleanUNet

Introduced in [39], CleanUNet tackles the task of **speech denoising**, aiming to enhance recorded speech by eliminating background noise without compromising perceptual quality and intelligibility. Unlike approaches relying on the frequency domain, CleanUNet

operates in the time domain, directly analyzing and manipulating the raw waveform samples. This enables real-time processing and avoids potential distortions introduced by domain transformations. Utilizing a U-Net structure with self-attention blocks, CleanUNet achieves superior performance in removing background noise while preserving speech intelligibility. This comprehensive analysis dissects the model’s architecture, training procedure, and results, demonstrating its effectiveness across various datasets and metrics.

6.2.1 Model Architecture

The baseline is a U-net architecture, which is composed of an encoder, a decoder, and a bottleneck between them. Both encoder and decoder have the same number of layers, D (Depth of the encoder). The encoder and decoder are connected by skip connections.

Each encoder layer consists of:

- Strided 1-d convolution (Conv1d): to down-sample the length of the input and increase the number of channels. Each Conv1d has kernel size K and stride $S = K/2$. The first layer’s conv1d outputs H channels, while the next layers double the number of input channels.
- ReLU (rectified linear unit).
- 1×1 convolution (Conv 1×1): to double the number of channels.
- Gated linear unit (GLU): given 2 inputs a and b , it outputs: $a * \text{sigmoid}(b)$. The number of channels is doubled by the Conv 1×1 because it’s halved by the GLU then.

Each decoder layer consists of:

- 1×1 convolution (Conv 1×1).
- GLU.
- Transposed 1-d convolution (ConvTranspose1d).

Each layer of the decoder is paired with the corresponding encoder layer in reversed order (via skip channels). The bottleneck consists of N self-attention blocks.

Each self-attention block consists of:

- Multi-head self-attention layer: contains 8 heads.
- Position-wise fully connected layer.

Each of these layers has a skip connection around it, then the layer is normalized.

The model dimension is 512. The attention map is masked, which means that during self-attention, each position in the sequence can only attend to positions earlier in the sequence, to ensure causality.

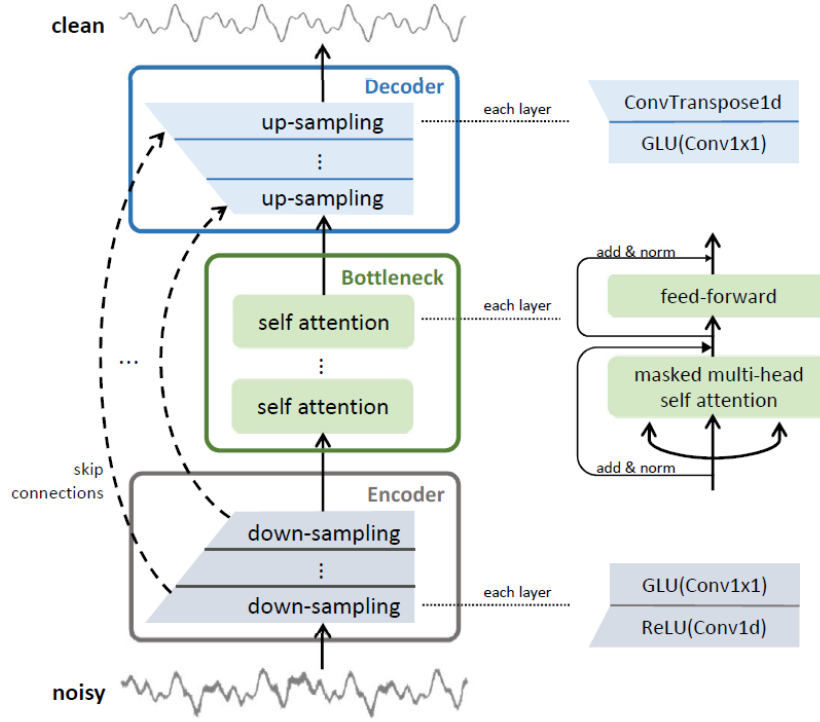


Figure 6.3: Model architecture for CleanUNet. It is causal with noisy waveform as input and clean speech waveform as output.

6.2.2 Experimental Setup

6.2.2.1 Datasets

The model is trained on three datasets: Deep Noise Suppression (DNS), Valentini, and an internal dataset. Each dataset provides aligned pairs of clean and noisy speech (x_{noisy}, x), with specific augmentation techniques employed for data enrichment.

6.2.2.1.1 DNS

It contains recordings of more than 10k speakers, reading 10k books with a sampling rate = 16kHz. 500 hours of clean and noisy speech pairs were synthesized. The length of each sample was 10 seconds.

The augmentation technique was RevEcho, which adds decaying echoes to the training data.

6.2.2.1.2 Valentini

It contains 28.4 hours of clean and noisy speech pairs, collected from 84 speakers, with a sampling rate = 48kHz. The length was between 1.33 and 1.5 seconds, randomly chosen. The augmentation techniques were:

- **Remix** : shuffles noises within a local batch.
- **BandMask** : removes a fraction of frequencies.

6.2.2.1.3 Internal Dataset

It contains 100 hours of clean and noisy speech pairs with a sampling rate = 48kHz. The length of each sample was 10 seconds. The augmentation technique was RevEcho, similar to DNS.

6.2.2.2 Training

Given 2 inputs, x and $x_{\text{noisy}} = x + x_{\text{noise}}$, this model should calculate $\hat{x} = f(x_{\text{noisy}}) \approx x$.

The loss function has 2 terms:

- L_1 loss on waveform.
- STFT loss: Short-time Fourier transform loss between x (clean speech) and $\hat{x}$ (denoised speech).

Before displaying these 2 terms, we must know some terms used in these 2 losses, which are: the Frobenius norm, L_1 norm, and M-STFT loss.

Frobenius norm $\|A\|_F$ is a matrix norm defined as the square root of the sum of the absolute squares of its elements, which is equivalent to the square root of the trace of A^*A .

L_1 norm $\|A\|_1$ is a matrix norm defined as the sum of the absolute values of its elements.

The STFT hyperparameters include the hop size, the window length, and the FFT bin. These hyperparameters are represented by ' θ '. Now, we will assume that $s(x, \theta) = |STFT(x)|$.

Multi-Resolution STFT loss (M-STFT) can be calculated as follows:

$$\text{M-STFT}(x, \hat{x}) = \sum_{i=1}^m \frac{\|s(x; \theta_i) - s(\hat{x}; \theta_i)\|_F}{\|s(x; \theta_i)\|_F} + \frac{1}{T} \|\log \frac{s(x; \theta_i)}{s(\hat{x}; \theta_i)}\|_1 \quad (6.2)$$

Then, after summing our 2 terms of loss, the whole loss function becomes:

$$\text{Loss} = \frac{1}{2} \text{M-STFT}(x, \hat{x}) + \|x - \hat{x}\|_1 \quad (6.3)$$

Using only the first term (full-band M-STFT) leads to low-frequency noises on the silence part while using the L_1 loss only results in less accurate results for the high-frequency bands. To resolve this problem, the full-band M-STFT was replaced with a high-band multi-resolution STFT loss $s_h(x)$ which contains only the second half number of rows of $s(x)$. Both full and high-band STFT losses are used in testing and comparing results.

6.2.3 Conclusion

In conclusion, the paper presents CleanUNet, a causal speech-denoising model in the waveform domain, which utilizes self-attention blocks to refine bottleneck representations. The proposed model outperforms the state-of-the-art models in terms of denoised speech quality from various objective and subjective evaluation metrics. The ablation studies conducted on the architecture and loss function demonstrate the effectiveness of the

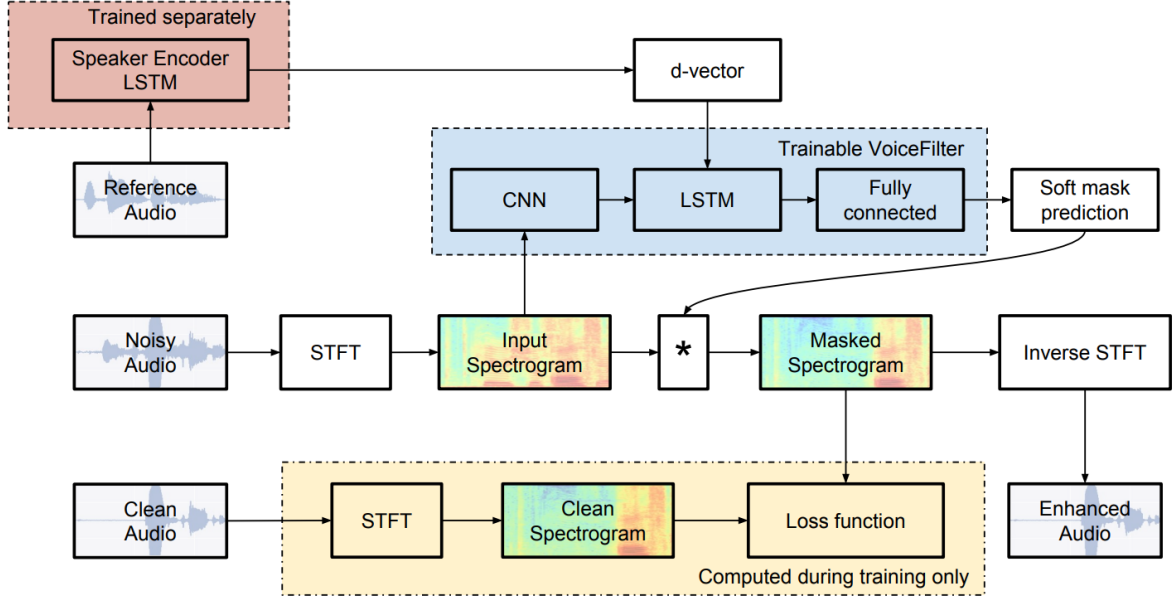


Figure 6.4: *VoiceFilter* architecture

proposed model. The proposed model can be used for online streaming applications and can be extended to other speech enhancement tasks.

6.3 VoiceFilter

VoiceFilter [40] aims to solve the issue of The Cocktail Party Problem, where there are multiple talkers speaking simultaneously, and you want only to listen to one person, which is relatively easy to do as a human being but hard to do as a machine. The main idea of this model is to separate the Target voice using spectrogram masking.

6.3.1 Model Architecture

The architecture has two models built in which are found in Figure 6.4:

1. Speaker Encoder (Red):
 - a. Input: An Utterance (a sound of the target speaker)
 - b. Output: Speaker-discriminative embedding
2. VoiceFilter (Blue):
 - a. Inputs:
 - i. Magnitude Spectrogram
 - ii. Speaker-discriminative embedding
 - b. Output: Mask

6.3.1.1 Speaker Encoder

The target speaker’s audio sample is fed into the speaker encoder, which creates a speaker embedding. This system is based on recent work by Wan et al. [41]. That delivers excellent results on speech-to-speech translation, multispeaker TTS, speaker diarization, and both text-dependent and text-independent speaker verification tasks.

A 3-layer LSTM network trained with the generalized end-to-end loss serves as the speaker encoder. It accepts log-mel filterbank energies that are taken out of 1600ms windows as inputs and outputs 256-dimensional speaker embeddings known as d -vectors. A sliding windows was used with 50% overlap and average the L_2 -normalized d -vectors that are produced on each window in order to compute a d -vector on a single utterance.

6.3.1.2 VoiceFilter

The recent work of Wilson et al. [42], created for voice augmentation, serves as the primary foundation for the VoiceFilter System. The target speaker’s d -vector and a magnitude spectrogram that was calculated from a noisy audio file are the two inputs that the neural network receives. An improved magnitude spectrogram is created by multiplying the soft mask that the network predicts element-wise by the input (noisy) magnitude spectrogram. The noisy audio is blended phase with the improved magnitude spectrogram to create the enhanced waveform, and then an inverse STFT is run on the output. The goal of the network’s training is to reduce the variation between the target magnitude spectrogram, which is calculated using the clean audio, and the masked magnitude spectrogram. Except for the final layer, which has a sigmoid activation, the VoiceFilter network is made up of eight convolutional layers, one LSTM layer, and two fully connected layers. All of these levels have ReLU activations. Table 6.1 presents the values of the parameters. In each time frame, the d -vector is concatenated to the output of the final convolutional layer many times. The subsequent LSTM layers receive the concatenated vector as their input. For two reasons, d -vector is injected the between the LSTM layer and the convolutional layers rather than before them. Firstly, d -vector doesn’t need to be altered by adding convolutional layers on top of it because it is already a compact and reliable representation of the target speaker. Second, convolutional layers cannot be applied to an input consisting of a magnitude spectrogram and a speaker embedding because they require time and frequency homogeneity. The input audios are split into 3-second parts for training the VoiceFilter system. If needed, they are then transformed to 16 kHz single-channel audios.

6.3.2 Experimental Setup

6.3.2.1 Dataset

Using two datasets concatenated, the MultiReader approach described in [41] was used to train the Speaker Encoder. Anonymized voice query records in English from mobile and far-field devices make up the first dataset. The dataset that comprises LibriSpeech [33], VoxCeleb, and VoxCeleb2 is the second one. Additionally, VoiceFilter was trained using data that was generated by the original authors and came from either the LibriSpeech or the VCTK dataset. Ten speakers were used for testing and 99 speakers for training in VCTK. They used the training and development sets—2338 speakers in the training set and 73 speakers in the development set—that were specified in the dataset’s procedure

Layer	Width		Dilation		Filters / Nodes
	time	freq	time	freq	
CNN 1	1	1	7	1	64
CNN 2	7	1	1	1	64
CNN 3	5	5	1	1	64
CNN 4	5	5	2	1	64
CNN 5	5	5	4	1	64
CNN 6	5	5	8	1	64
CNN 7	5	5	16	1	64
CNN 8	1	1	1	1	8
LSTM	-	-	-	-	400
FC 1	-	-	-	-	600
FC 2	-	-	-	-	600

Table 6.1: *Parameters of the VoiceFilter network.*

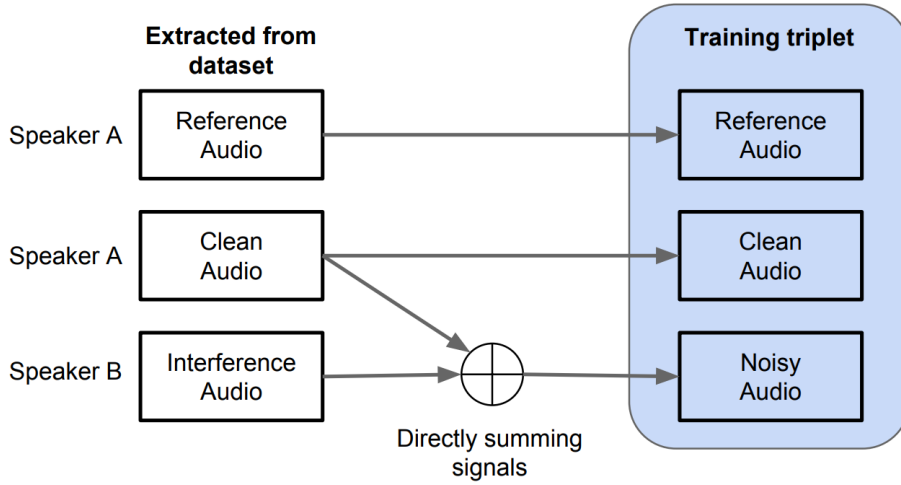


Figure 6.5: *Input data processing workflow.*

for LibriSpeech. Each utterance in these two read speech datasets features the voice of a single speaker. And the data generation flow is shown in Figure 6.5.

6.3.2.2 Evaluation

To evaluate the performance of different VoiceFilter models, Two metrics were used : the speech recognition Word Error Rate (WER) and the Source to Distortion Ratio (SDR).

6.3.2.2.1 Word Error Rate (WER)

Four WER numbers were important to them:

- **Clean WER:** The WER on the clean audio, without VoiceFilter.
- **Noisy WER:** The WER on the noisy (clean + interference) audio in the absence of VoiceFilter.
- **Clean-enhanced WER:** The WER on the clean audio that the VoiceFilter system

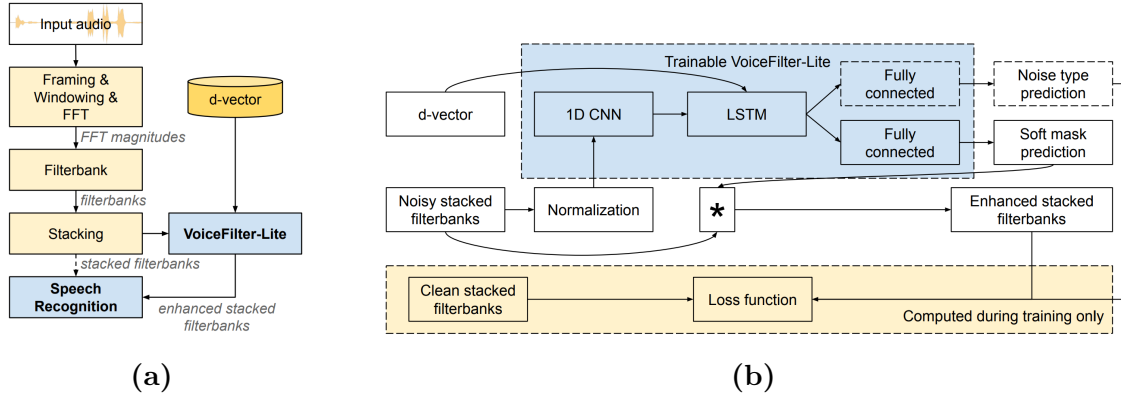


Figure 6.6: *VoiceFilter-Lite architecture, assuming using stacked filterbank energies as inputs and outputs. (a) Integration with ASR. The dashed arrow indicates the original connection without VoiceFilter-Lite. (b) Neural network topology of the VoiceFilter-Lite model.*

processed.

- **Noisy-enhanced WER:** The WER on the loud audio that the VoiceFilter system processed.

These two characteristics should be present in a decent VoiceFilter model:

- Since noisy-enhanced WER is much lower than noisy WER, the VoiceFilter is helping with multi-speaker speech recognition.
- Clean-enhanced WER is quite close to Clean WER, indicating that in single-speaker circumstances, the VoiceFilter has few adverse effects.

6.3.2.2.2 Source-to-Distortion Ratio (SDR)

A popular statistic for assessing source separation systems is the SDR, which necessitates knowledge of both the augmented and clean signals. It is a dB-expressed energy ratio that compares the energy of the target signal—which is contained in the boosted signal—to the energy of the mistakes, which are caused by artifacts and interfering speakers. Therefore, the higher, the better.

6.4 VoiceFilter-Lite

VoiceFilter-Lite [43] is one of the future versions of VoiceFilter [40], which can be summarized as a single channel source separation model for a target speaker, which is mainly a part of on-device streaming ASR (Automatic Speech Recognition), where takes filterbanks as inputs and outputs, and it is a quantized 8-bit model to reduce its size.

6.4.1 Model Architecture

Figure 6.6a shows how the VoiceFilter-Lite can be integrated with the ASR System, while Figure 6.6b shows the actual model topology of VoiceFilter-Lite where it is similar to the Original VoiceFilter but with a few differences to guarantee minimal latency:

- Limited convolution layers to be 1D instead of 2D, where the kernels are now in the frequency dimension only.
- LSTM layers must be unidirectional and must take streaming inputs.

Since 1D-CNN is not as powerful as 2D-CNN, the model mainly consists of LSTM layers.

6.4.1.1 Model Quantization

The network parameters are stored in 32-bit floating point values in raw TensorFlow graphs, which are not well suited for on-device inference. The models are serialized to the FlatBuffer TensorFlow Lite format and use dynamic range quantization to quantize the network parameters to 8-bit integers. This allows us to better utilize the improved hardware instructions for integer arithmetic and drastically lower the cost of memory.

6.4.2 Experimental Setup

Since the goal of the VoiceFilter-Lite model is to enhance on-device ASR, our statistic of choice is the Word Error Rate (WER). In particular, they were concerned about the WER in different noise situations both before and after using VoiceFilter-Lite. Given the emphasis on speech recognition and the fact that the VoiceFilter-Lite model does not reconstruct any waveform from the enhanced acoustic features, metrics like signal-to-noise ratio (SNR) or source-to-distortion ratio (SDR) that have been widely used for conventional source separation were not reported.

6.4.2.1 Model and Data

In the experiments, the VoiceFilter-Lite model has 3 LSTM layers, each with 512 nodes, and a final fully connected layer with sigmoid activation. If noise type prediction is enabled, it has another two feedforward layers, each with 64 nodes, and is trained with a hinge loss for binary classification. The training data consist of:

1. The LibriSpeech [33] training set.
2. A vendor collected a dataset of realistic speech queries.

The study evaluates data using various noise sources and room configurations. The original non-modified data is referred to as "Clean", while the speech noise source is a distinct development set. The evaluations are conducted under two room conditions: "Additive" which adds noise waveforms to the clean waveform, and "Reverb" which generates 3 million convolutional room impulse responses (RIR) from a room simulator. The SNR is drawn from a uniform distribution from 1dB to 10dB for both additive and reverberant conditions.

6.4.3 Conclusion

VoiceFilter-Lite is a fast, tiny model that performs targeted voice separation in a streaming fashion for on-device ASR systems. It ensures ASR performance without affecting non-speech noise and reverberant room conditions. The model is trained with an asymmetric loss function and adaptive suppression strength at run-time. The 2.2 MB model

has no WER degradation on clean and non-speech noise conditions while improving ASR performance on overlapped speech.

6.5 Inter-Subnet

Chen et al. [44] took the subband approach to solve the single-channel speech enhancement problem, building upon the work of Hao et al.[45]. The authors of the paper proposed a subband interaction module to capture the inter-subband dependencies with a lower number of parameters. InterSubNet is a novel, lightweight framework for speech enhancement. It alternates between a SubInter module and a Long Short-Term Memory (LSTM) layer. This design allows it to grasp the overall spectral information while maintaining its capacity to concentrate on local spectral patterns.

6.5.1 Model Architecture

Considering the short-time Fourier transform STFT of a single-channel signal with 16 kHz sampling rate:

$$X_{t,f} = S_{t,f} + N_{t,f} \quad (6.4)$$

where $S_{t,f}$ is the clean speech and $N_{t,f}$ is the noise. The network is trained to estimate the clean speech $S_{t,f}$ from the noisy speech $X_{t,f}$.

The complete structure of Inter-SubNet G_{is} is illustrated in Figure 6.7. It takes as input a magnitude spectrogram $X \in \mathbb{R}^{F \times T}$. Here, F and T denote the total count of frequency bins and frames, respectively. The first step transforms the spectrogram into subband units $\{\mathbf{b}_i\}_{i=1}^F$ using an "unfold" operation. These subband units are then input into the G_{is} model, which generates the final outputs cIRM M^r and M^i . The main structure of G_{is} includes two stacked SubInter-LSTM (SIL) blocks, followed by a single fully connected layer.

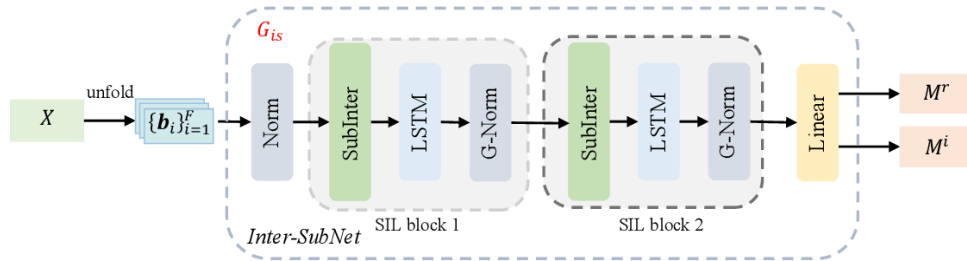


Figure 6.7: The general schematic of the proposed Inter-SubNet, where "SIL block 1" and "SIL block 2" refer to the first and second SubInterLSTM block respectively. The "G-norm" denotes group normalization [46]. The Inter-SubNet G_{is} mainly consists of two stacked SIL blocks and one fully connected layer. Taking the subband units $\mathbf{b}_{i=1}^F$ as input, the model G_{is} generates the final output cIRM M^r and M^i .

6.5.1.1 Subband units

Subband units are derived from the magnitude spectrogram X using the "unfold" operation and are used as the input for the Inter-SubNet. Specifically, for each frequency

i from the magnitude spectrogram X , a frequency bin vector $X_i \in \mathbb{R}^T$ is taken, along with the $2n$ frequency bin vectors adjacent to it, to form a subband unit b_i . This can be represented as:

$$b_i = [X_{i-n}, \dots, X_i, \dots, X_{i+n}] \in \mathbb{R}^{Fs \times T} \quad (6.5)$$

Here, "[,]" signifies the concatenation operation and $Fs = 2n + 1$. Additionally, for boundary frequencies where $i + n > F - 1$ or $i - n < 0$, circular Fourier frequencies are assigned. Through this process, F subband units are obtained, each with a dimension of (Fs, T) . These are denoted as $\{\mathbf{b}_i\}_{i=1}^F \in \mathbb{R}^{F \times Fs}$.

6.5.1.2 SubInter module

Inspired by [47], The SubInter module is designed to extract global spectral information, which includes dependencies across different bands and overall spectral patterns. This is achieved through the interaction among the parallel subband units. Additionally, this module integrates the extracted global spectral information back into these subband units. Having a collection of F subband units $\{b_i\}_{i=1}^F$, where each $b_i \in \mathbb{R}^{Fs \times T}$ and Fs denotes the number of frequency bins in a subband unit. The SubInter module operates on these subband units to incorporate complementary global spectral information:

$$\{\hat{\mathbf{b}}_i\}_{i=1}^F = H(\{\mathbf{b}_i\}_{i=1}^F) \quad (6.6)$$

Here, $\hat{\mathbf{b}}_i \in \mathbb{R}^{Fs \times T}$ is the subband feature that encapsulates both global spectral information and local spectral patterns, and $H(\cdot)$ is the mapping function that characterizes the SubInter module.

6.5.2 Experimental Setup

6.5.2.1 Dataset

The authors used a subset of the DNS Challenge - Interspeech 2021 dataset for training. Room impulse responses were randomly sampled and added to 75% of the clean speech. Afterward, the noisy speeches were mixed with the clean or reverberant speeches at a randomly selected SNR varying from -5 to 20 dB.

6.5.2.2 Training

STFT with a Hanning window with a length of 32 ms and a hop size of 16 ms was used to obtain the magnitude spectrogram. The model was trained for 1000 epochs with the Adam optimizer with a learning rate $1e^{-3}$, a batch size of 40 (2 GPUs with 20 samples each), and MSE loss. The authors used subband units with $n = 15$ (i.e., 15 neighbor frequencies are taken on each side of each input frequency bin). In the training procedure, the input-target sequence pairs were generated into a constant length sequence of $T = 192$ frames (approximately 3 s).

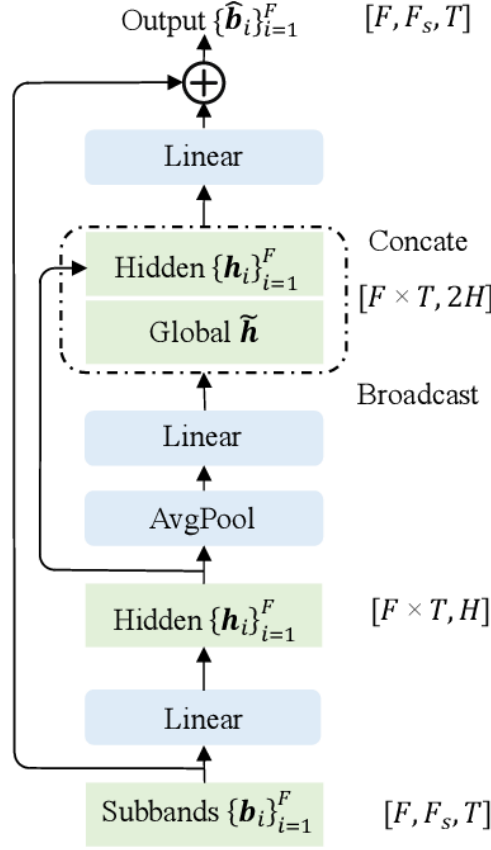


Figure 6.8: The details of the SubInter Module. The “Hidden” indicates the hidden representations. The “Global” denotes the output of the second linear layer containing global spectral information.

6.5.3 Conclusion

This paper presents the Subband Interaction (SubInter) module, a novel approach to modeling global spectral information for speech enhancement, and introduces a new framework called Inter-SubNet. This innovative, lightweight framework for speech enhancement is engineered to leverage global spectral information while maintaining its capacity to concentrate on local spectral patterns.

6.6 Fast FullSubNet

Subband approaches have been widely used in speech enhancement tasks, as they allow the model to focus on local spectral patterns. Many variants of the subband approach have been proposed, focusing on improving the model’s ability in noise reduction and speech enhancement. However, these approaches often suffer from high computational complexity and slow inference speed. To address this issue, Hao and Li[2] proposed a novel, lightweight framework called Fast FullSubNet. Fast FullSubNet is a subband-based speech enhancement model that is designed to be fast and efficient while maintaining high performance in noise reduction tasks. It utilizes the mel-frequency domain to reduce the number of frequency bands while maintaining the spectral information. The Mel-frequency representation effectively condenses speech spectra while concurrently

maintaining spectral information crucial for human auditory perception.

6.6.1 Model Architecture

The Fast FullSubNet model consists of three main components: Full-band model F_{l2m} (encoder), Sub-band model S (bottleneck) and Full-band model F_{m2l} (decoder). Figure 6.9 illustrates the architecture of Fast FullSubNet.

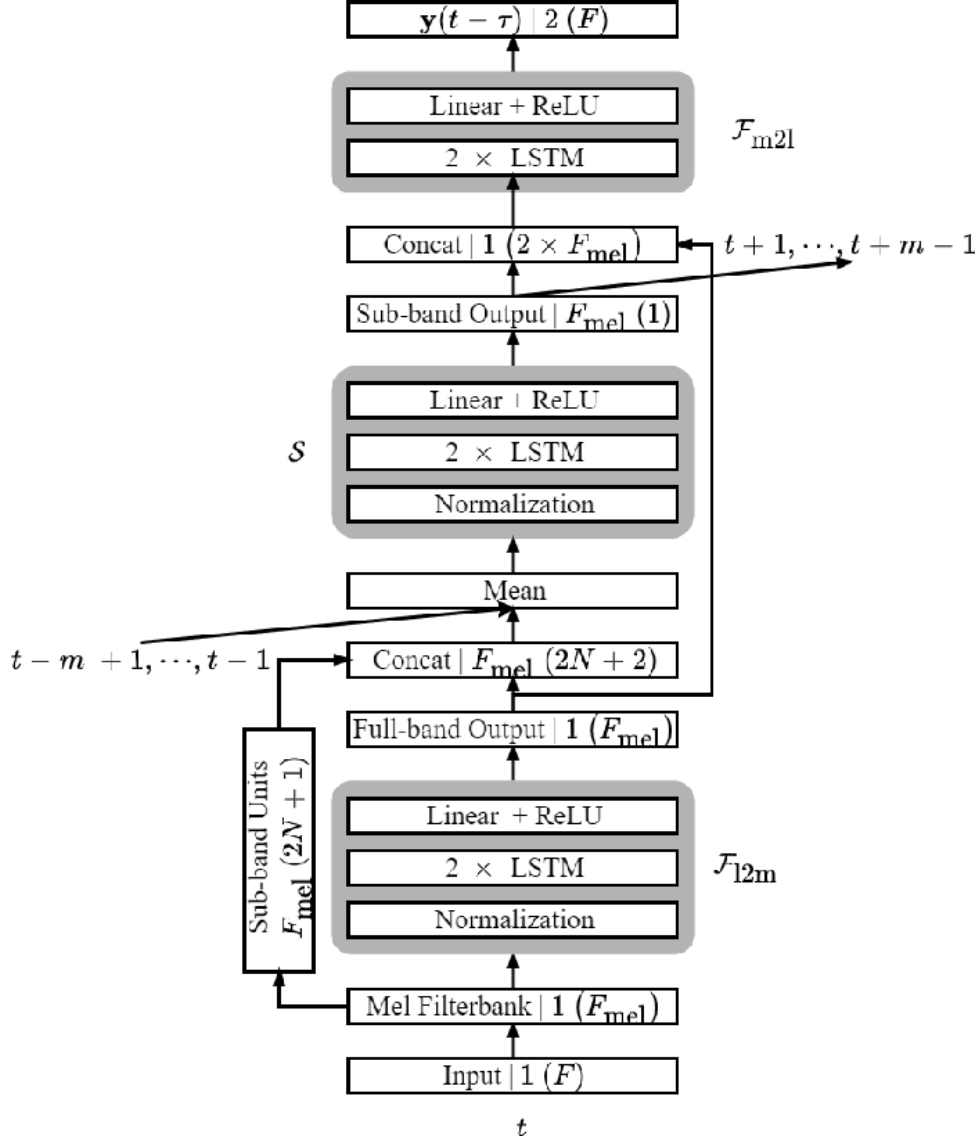


Figure 6.9: Diagram of Fast FullSubNet. The right parts of rectangle boxes show feature dimensions, e.g., “ $1 (F)$ ” represents a F -dimension vector. “ $F_{mel}(2N + 2)$ ” denotes F_{mel} independent $(2N + 1)$ -dimensional vectors.

6.6.1.1 Encoder

The mel-scale spectral magnitude undergoes initial processing by the full-band model, F_{l2m} , which extracts global spectral characteristics and long-range dependencies across bands. Specifically, the linear frequency signal $x(t, f)$ is transformed into the mel-

frequency domain as $x_{\text{mel}}(t, f)$, where $f \in [0, \dots, F_{\text{mel}} - 1]$ and F_{mel} denotes the number of mel frequencies. The input vector to Fl2m at time t is represented as

$$x(t) = [x_{\text{mel}}(t, 0), \dots, x_{\text{mel}}(t, F_{\text{mel}} - 1)]^T \in \mathbb{R}^{F_{\text{mel}}}$$

This feature vector sequence is then processed by two layers of LSTM. The full-band model produces a spectral embedding of the same dimension as $x(t)$, with each mel-frequency corresponding to one hidden unit. This spectral embedding complements the subsequent sub-band model by providing additional informative features.

6.6.1.2 Bottleneck

The sub-band model operates on mel-frequencies independently within a shared network structure. It predicts clear speech by leveraging the stationary nature of signals and the local spectral patterns specific to speech. Each input to the sub-band model incorporates two main sources. Firstly, for a given frequency f , the input consists of the noisy mel-spectra at that frequency and N adjacent frequencies on each side. Secondly, it includes the output of the full-band model at frequency f , denoted as $\text{Fl2m}(x(t))(f)$. These sources are concatenated to form the input vector

$$x_{\text{sub}}(t, f) = [x_{\text{mel}}(t, f - N), \dots, x_{\text{mel}}(t, f + N), \text{Fl2m}(x(t))(f)]^T \in \mathbb{R}^{2N+2} \quad (6.7)$$

This feature vector sequence is uniformly processed using two LSTM layers across all frequencies. Each mel-frequency processed by the sub-band model yields a single-dimensional hidden unit output. To accelerate inference speed, the feature sequence is downsampled by a factor of m before being processed by the bottleneck, exploiting redundancies among successive samples.

6.6.1.3 Decoder

The full-band model Fm2l reverses the mel-frequency transformation to obtain linear-frequency representations. It combines the outputs of the full-band model Fl2m and the sub-band Model S into a concatenated input vector $x_{\text{m2l}}(t)$ for Fm2l:

$$x_{\text{m2l}}(t) = [\text{Fl2m}(x(t))^T, S(x_{\text{sub}}(t, 0)), \dots, S(x_{\text{sub}}(t, F_{\text{mel}} - 1))]^T \in \mathbb{R}^{2F_{\text{mel}}} \quad (6.8)$$

This input is processed by two layers of LSTM followed by a linear layer to predict the final linear-frequency output. The model aims to predict the real-valued complex-valued Ideal Ratio Mask (cIRM) $y(t)$ for each time-frequency bin:

$$y(t) = [R\{y(t, 0)\}, I\{y(t, 0)\}, \dots, R\{y(t, F - 1)\}, I\{y(t, F - 1)\}]^T \in \mathbb{R}^{2F} \quad (6.9)$$

Here, R and I denote the real and imaginary parts of the complex number, respectively. This mel-to-linear model functions similarly to neural vocoders used in Text-to-Speech (TTS) applications, converting mel-frequency representations to linear-frequency representations, with the capability to enhance speech using noisy signal phases.

6.6.2 Experimental Setup

6.6.2.1 Dataset

To ensure a fair comparison with FullSubNet, experiments were conducted on the DNS challenge dataset [29] from INTERSPEECH 2020. The training involved synthesizing

noisy-clean speech pairs using dynamic mixing strategies similar to FullSubNet. Before each epoch, 75% of clean speech clips were mixed with room impulse responses from two databases, and combined with noise at varying SNR levels (-5 dB to 20 dB). Training over ten epochs exposed the model to over 5000 hours of data. The evaluation used the DNS Challenge test dataset, consisting of 150 noisy clips each with and without reverberation, and SNRs ranging from 0 dB to 20 dB.

6.6.2.2 Training

The audio signals are sampled at 16000 Hz, and their Short-Time Fourier Transform (STFT) uses a 32 ms (512 samples) Hanning window with a 16 ms hop size. Mel-frequency analysis employs 64 bins. Training utilizes the Adam optimizer with a learning rate of 0.001, and to maintain consistency with prior work, a two-frame output delay (equating to 32 ms) is implemented to allow the model to utilize future information effectively. The training sequence length is set at $T = 192$ frames (about 3 seconds). Mean Squared Error (MSE) is employed as the loss function throughout the study.

6.6.3 Conclusion

Fast FullSubNet is a lightweight, efficient subband-based speech enhancement model that maintains high performance in noise reduction tasks. The model utilizes the mel-frequency domain to reduce the number of frequency bands while preserving spectral information. The Fast FullSubNet model consists of three main components: Full-band model F_{l2m} (encoder), Sub-band model S (bottleneck), and Full-band model F_{m2l} (decoder). The model was trained on the DNS Challenge dataset from INTERSPEECH 2020, and evaluation was conducted on the DNS Challenge test dataset.

6.7 TitaNet

For extracting speaker representations (embeddings), Koluguri et al. [48] proposed a novel network architecture called TitaNet. TitaNet simply takes a variable-length speech signal as input and outputs a fixed-length speaker embedding. Their architecture achieves state-of-the-art performance on both speaker verification and speaker diarization tasks.

6.7.1 Model Architecture

The model is based on the encoder and decoder structure. The encoder is used for feature extraction and then those features are passed to the attention-pooling layer in order to calculate attention features across channel dimensions.

The encoder-decoder architecture of TitaNet is shown in Figure 6.10.

6.7.1.1 Encoder

The model encoder is based on ContextNet- $B \times R \times C$ model such that:

- B : number of blocks
- R : number of sub-blocks per block

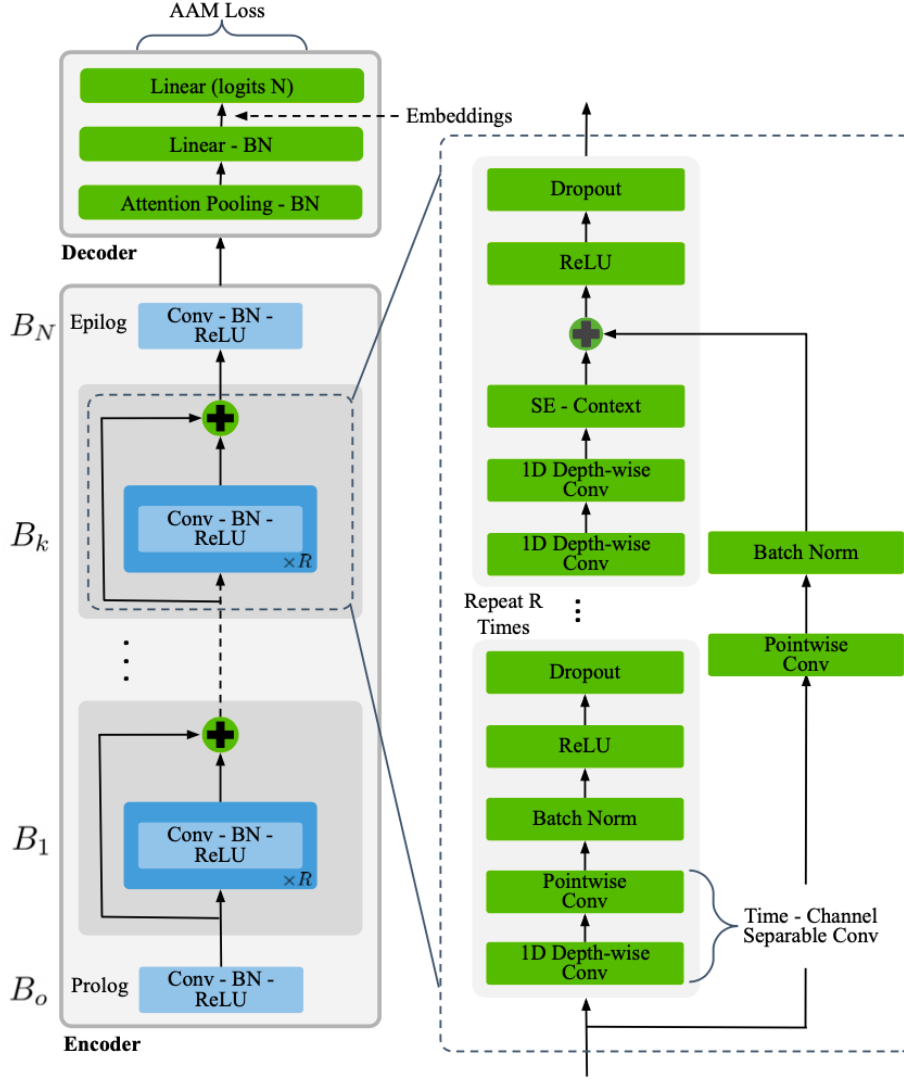


Figure 6.10: *TitaNet Encoder and Decoder Architecture*

- C : number of filters in the convolution layers of each block

The encoder starts with prologue block B_0 followed by $N - 2$ mega blocks and finally, it ends with epilogue block B_N . Both prologue and epilogue blocks are different from other mega blocks; they have a convolution layer followed by batch normalization and ReLU layers but they differ in kernel size (3 and 1 for prologue and epilogue respectively). As for the mega blocks, they consist of time-channel separable convolution (with stride 1 and dilation 1), followed by batch normalization, ReLU, and dropout. Those layers are repeated R times to allow varying depth. In the global context, these channels are residually connected with the Squeeze and Excitation layer.

6.7.1.2 Decoder and Embeddings

The high-level features computed by the encoder are then passed to the decoder to get the speaker embedding. The attentive statistics pooling layer is used to get a time-independent feature representation S of size $B \times 3072$. These features are then passed to 2 linear layers:

- l_1 : Output a fixed-size embedding of size 192
- l_2 : Output a vector of probabilities that the given utterance belongs to one of the training set speakers

6.7.1.3 Loss function

TitaNet uses additive angular margin (AAM) loss. It helps to optimize the cosine distance between speaker embeddings:

$$L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s(\cos(\theta_{y_i}+m))}}{e^{s(\cos(\theta_{y_i}+m))} + \sum_{j=1, j \neq y_i}^n e^{s \cos \theta_j}} \quad (6.10)$$

where m is margin, s is scale and θ_j is the angle between the final layer weight W_j and incoming feature x_i . Both m and s are predefined hyperparameters.

6.7.2 Experimental Setup

Three TitaNet models were designed: TitaNet-S, TitaNet-M, and TitaNet-L with 256, 512, and 1024 channels respectively. All the mentioned models have the same R as 3 and the same filter sizes as 3, 7, 11, and 15.

6.7.2.1 Dataset

The authors used VoxCeleb1[49], VoxCeleb2[50] dev, LibriSpeech[33] as well as other datasets with a total number of speakers of 16681 and a duration of 3373 hours. The training data is also augmented using RIR impulse corpora as well as different speed perturbations (0.95x and 1.05x).

6.7.3 Conclusion

This paper presents TitaNet, a novel model for speaker representation learning that uses the global context of SE layers along with channel attentive statistics pooling for the extraction of fixed-length speaker embedding. All model implementations and checkpoints are available in the NeMo toolkit (provided by NVIDIA).

6.8 ECAPA-TDNN

Desplanques et al. [51] proposed ECAPA-TDNN architecture that significantly outperformed state-of-the-art TDNN-based systems on the VoxCeleb test sets and the 2019 VoxCeleb Speaker Recognition Challenge [52]. Notably, the architecture was also recommended by the Deep Noise Suppression challenge in 2023 [1] for its robustness and efficiency in speaker embedding tasks. It was also used by the state-of-the-art TEA-PSE 3.0 [53] model which won the first place in the 2023 Deep Noise Suppression Challenge.

The authors proposed several enhancements to the successful Time Delay Neural Network (TDNN) architecture, drawing on recent advancements in face verification and computer vision. Traditionally, TDNN uses statistics pooling to convert variable-length utterances into fixed-length speaker-characterizing embeddings.

Firstly, the initial frame layers were restructured into 1-dimensional Res2Net modules [54] with impactful skip connections. These modules also incorporate Squeeze-and-Excitation (SE) blocks [55], similar to those in SE-ResNet, to explicitly model channel interdependencies. The SE blocks expand the temporal context of the frame layer by rescaling the channels based on the recording's global properties.

Secondly, recognizing that neural networks learn hierarchical features, with each layer capturing different levels of complexity, the authors aggregated and propagated features from various hierarchical levels to leverage this complementary information.

Finally, they enhanced the statistics pooling module with channel-dependent frame attention. This improvement allows the network to focus on different subsets of frames during each channel's statistics estimation.

6.8.1 Model Architecture

An overview of the complete architecture can be shown in Figure 6.12.

6.8.1.1 Context-dependent statistics pooling

Recent x-vector architectures employed soft self-attention for calculating weighted statistics in the temporal pooling layer. The success of multi-headed attention demonstrated that specific speaker properties could be extracted from different sets of frames. Building on these results, the authors proposed extending this temporal attention mechanism to the channel dimension. This extension allowed the network to focus on speaker characteristics that manifested at different time instances, such as the distinct properties of vowels versus consonants.

6.8.1.2 Res2Blocks

The temporal context of frame layers in the original x-vector system was limited to 15 frames. Recognizing that a wider temporal context could be beneficial, the authors suggested rescaling frame-level features based on the global properties of the recording, similar to the global context in the attention module described above. For this purpose, the authors introduced 1-dimensional Squeeze-Excitation (SE) blocks, a technique proven successful in modeling global channel interdependencies in computer vision.

Integrating the 1-dimensional SE block into the x-vector architecture could be done in various ways, with the most straightforward method being their placement after each dilated convolution. However, the authors aimed to combine these blocks with the advantages of residual connections while minimizing the increase in total parameters compared to baseline systems. The SE-Res2Block, depicted in Figure 6.11, met these requirements. It included dilated convolutions flanked by dense layers with a context of 1 frame each. The first dense layer reduced the feature dimension, while the second restored it to the original dimension. This was followed by a SE-block that scaled each channel, with the entire unit enclosed by a skip connection.

The use of these traditional ResBlocks makes it easy to incorporate advancements concerning this popular computer vision architecture. The recent Res2Net module, for example, enhances the central convolutional layer so that it can process multi-scale features by

constructing hierarchical residual-like connections within. The integration of this module improved the performance, while significantly reducing the number of model parameters.

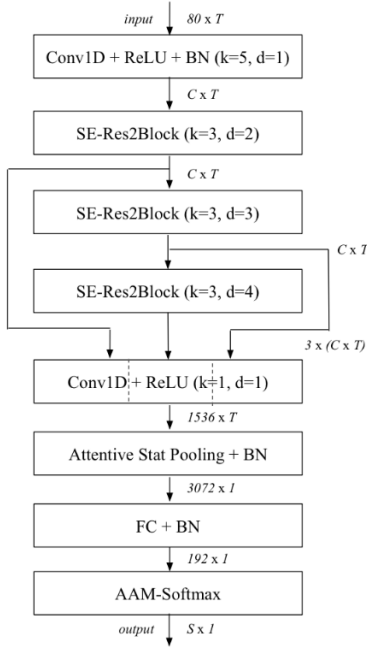


Figure 6.11: The *SE-Res2Block* of the *ECAPA-TDNN* architecture.

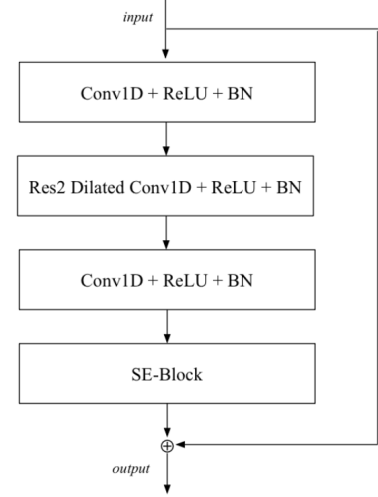


Figure 6.12: Network topology of the *ECAPA-TDNN*.

6.8.1.3 Hierarchical feature aggregation

The original x-vector system only uses the feature map of the last frame layer for calculating the pooled statistics. Given the hierarchical nature of a TDNN, these deeper-level features are the most complex ones and should be strongly correlated with the speaker identities. However, the authors argued that the more shallow feature maps can also contribute towards more robust speaker embeddings. For each frame, their proposed system concatenates the output feature maps of all the SE-Res2Blocks. After this Multi-layer Feature Aggregation (MFA), a dense layer processes the concatenated information to generate the features for attentive statistics pooling.

Another, complementary way to exploit multi-layer information is to use the output of all preceding SE-Res2Blocks and the initial convolutional layer as input for each frame layer block. The authors implemented this by defining the residual connection in each SE-Res2Block as the sum of the outputs of all the previous blocks. They opted for a summation of the feature maps instead of concatenation to restrain the model parameter count. The final architecture without the summed residual connections is shown in Figure 6.12.

6.8.2 Experimental Setup

The training was done using the development set of the VoxCeleb2 dataset with a split of 98% for training data and reserving a small subset of 2% as a validation set for hyper-

parameter optimization. Various data augmentation techniques were introduced to the VoxCeleb2 dataset, including:

- Kaldi recipe [56]
- Babble effect and extra noises from MUSAN dataset [57]
- Reverberation effect
- Open-source SoX library (tempo up, tempo down)
- FFmpeg library for alternating opus and acc compression

The input features are 80-dimensional MFCCs from a 25 ms window with a 10 ms frameshift. Two second random crops of the MFCCs feature vectors are normalized through cepstral mean subtraction and no voice activity detection is applied.

The models are trained using a batch size of 128 and with a learning rate varying between 1×10^{-8} and 1×10^{-3} using the triangular2 policy and Adam optimizer [37]. Each cycle has 130k iterations. To prevent overfitting, weight decay on all weights was applied with 2×10^{-5} , except for the AAM-softmax weights, which used 2×10^{-4} .

The dimension of the bottleneck in the SE-Block and the attention module is set to 128. The scale dimension in the Res2Block is set to 8. The number of nodes in the final fully-connected layer is 192.

Speaker embeddings are extracted from the final fully connected layer for all systems. Trial scores are produced using the cosine distance between embeddings. Subsequently, all the scores are normalized using adaptive s-norm.

6.8.3 Conclusion

In conclusion, the ECAPA-TDNN paper introduces a novel speaker embedding extractor that significantly improves speaker verification performance over strong baseline systems. By incorporating advanced features such as Res2Net modules, Squeeze-and-Excitation blocks, and multi-layer feature aggregation, the ECAPA-TDNN architecture demonstrates a 19% average relative improvement in EER on the VoxCeleb and VoxSRC 2019 evaluation sets. The inclusion of skip connections for channel propagation and attentive statistics pooling further enhances the model's ability to capture speaker identities accurately. These architectural enhancements showcase the potential for leveraging global context information and multi-layer features in TDNN-based speaker recognition systems, paving the way for more robust and efficient speaker verification technologies.

Chapter 7

Models

This chapter discusses the progress and experiments of choosing the model.

7.1 Project Stack

Different tools and libraries were considered for the project. The following is a list of the tools and libraries that were chosen for the project.

7.1.1 PyTorch

PyTorch is an open-source machine-learning library based on the Torch library. It's primarily developed by Facebook's artificial intelligence research group and is used for applications such as computer vision and natural language processing.

It is a popular choice for deep learning and AI projects due to several reasons:

1. **Dynamic Computation Graphs:** Unlike other libraries that have static computation graphs, PyTorch allows you to change the way your graph behaves on the fly. This is particularly useful for models where you need to change the structure during training.
2. **Pythonic Nature:** PyTorch's design embraces the Python philosophy, making it more intuitive and easier to use for Python developers. It integrates well with the Python data science stack, including libraries like NumPy and Matplotlib.
3. **Strong GPU Support:** PyTorch provides seamless CUDA integration for tensor computations, allowing you to easily compute on GPUs.
4. **Active Community and Good Documentation:** PyTorch has a very active community that contributes to a rapidly expanding ecosystem of tools and libraries. It also has excellent documentation and tutorials, making it accessible for beginners.
5. **Research and Production:** PyTorch is not only good for prototyping and research due to its flexibility, but it also has strong support for model deployment in production. This makes it a comprehensive tool for machine learning projects.

6. **Debugging:** As PyTorch is deeply integrated into Python, you can use any Python debugger with it. This makes debugging your models and training loops much easier compared to other libraries.

7.1.2 PyTorch Lightning

PyTorch Lightning is a lightweight PyTorch wrapper for high-performance AI research. It structures your code, so it's decoupled into four sections: Research code (the Lightning-Module), Engineering code (Trainer), Non-essential research code (callbacks), and Data (DataLoaders). This allows each part to be testable, reusable, and scalable independently.

It offers some key features:

1. **Simplicity:** PyTorch Lightning reorganizes your PyTorch code and decouples the science code from the engineering code which makes your code much cleaner and easier to understand.
2. **Compatibility:** PyTorch Lightning is just organized PyTorch. You can use all the PyTorch functions and modules you're familiar with transparently directly within PyTorch Lightning.
3. **Scalability:** PyTorch Lightning has built-in support for multi-GPUs, TPUs, and even scaling across clusters of machines.
4. **Advanced features:** PyTorch Lightning also provides advanced features like 16-bit precision, gradient accumulation, and more.
5. **Community and Ecosystem:** PyTorch Lightning has a growing community and ecosystem, with a variety of projects, tools, and libraries built around it.

7.1.3 Hydra

Hydra is an open-source Python framework that simplifies the development of research and complex applications. The key feature of Hydra is the ability to dynamically create a hierarchical configuration by composition and override it through config files and the command line. The name "Hydra" refers to its ability to manage complex, multi-headed systems.

Hydra provides the following features:

1. **Configuration Composition:** Hydra allows you to combine multiple configuration files, enabling you to easily compose configurations for your different use cases.
2. **Configuration Override:** You can override your configuration through the command line, making experimentation easy.
3. **Plug-ability:** Hydra is designed to be easy to extend and integrate with other systems. You can develop plugins to extend Hydra's capabilities.
4. **Applications:** Hydra can help manage complex applications and systems, making it easier to handle multi-step workflows, manage environments, and more.

Hydra is commonly used in machine learning research, where managing complex experiments and workflows is a common task.

7.1.4 Weights and Biases

Weights & Biases (wandb) is a tool designed to help track experiments in machine learning. It helps to log and visualize metrics from your runs, organize your experiments, and share findings with colleagues.

Wandb is useful in:

1. **Experiment Tracking:** You can track and visualize all the pieces of your machine learning model — hyperparameters, metrics, model weights, and more.
2. **Reproducibility:** Weights & Biases help ensure your models are reproducible by automatically capturing all the relevant information needed to replicate a run.
3. **Collaboration:** You can share your findings, models, and raw data with your team, enabling better collaboration.
4. **Integration:** Weights & Biases integrates seamlessly with popular machine learning frameworks like TensorFlow, PyTorch, Keras, and others.
5. **Dashboard:** It provides a flexible and intuitive dashboard to visualize your metrics, compare different runs, and see your model's performance over time.

7.1.5 TensorBoard

TensorBoard helps in logging and visualizing various types of data, such as scalars, images, audio, histograms, text, and more. This is particularly useful in machine learning experiments where tracking and visualizing metrics, model performance, and other data can help in model development and debugging.

Here's a brief introduction to some key aspects of TensorBoard logging:

1. **Scalars:** Scalars like loss, accuracy, learning rate, or other metrics can be logged and visualized over training steps or epochs.
2. **Images:** If you're working with image data or image-based models, you can log and visualize images, which can include input data, model predictions, or even model weights visualized as images.
3. **Audio:** Audio data can be logged and played back in the TensorBoard interface.
4. **Histograms:** Histograms allow you to visualize the distribution of values in a tensor over time. This can be useful for understanding how weights or other data are changing over the course of training.
5. **Graphs:** You can visualize your model's computational graph, which can help understand and debug the model.
6. **Embeddings:** High-dimensional data like embeddings can be visualized and explored interactively.
7. **Text:** Text data can be logged and visualized, which can be useful for tasks like natural language processing.

7.2 Previous Experiments

To get a feel of the training procedure, the learning curve, and the potential outputs, we ran several training experiments on different models and datasets. The experiments were run using Kaggle’s GPU kernels, which provide a free GPU for training. The training procedures and the results are discussed below.

7.2.1 CleanUNet

We tried to replicate the training procedure stated in [39] for the CleanUNet model. We used 100 hours of the DNS 2020 dataset [29] for training, and its corresponding test set was used for testing.

Architecture: The depth of the encoder or the decoder is $D = 8$. Each encoder/decoder layer has hidden dimension $H = 64$, stride $S = 2$, and kernel size $K = 4$. The depth of the bottleneck is $N = 5$. Each self-attention block has 8 heads, model dimension = 512, middle dimension = 2048, no dropout, and no positional encoding.

Optimization: The model was trained for 50k steps with a batch size of 4. The Adam [37] optimizer was used with a learning rate of 2×10^{-4} with a Linear Warmup Cosine Decay. The loss functions used were L1 loss and STFT loss [3]. The model was trained on 2 NVIDIA Tesla T4 GPUs. The training took 24 hours to complete. The training loss is shown in Figure 7.1 and metrics on the test set are shown in Table 7.1.

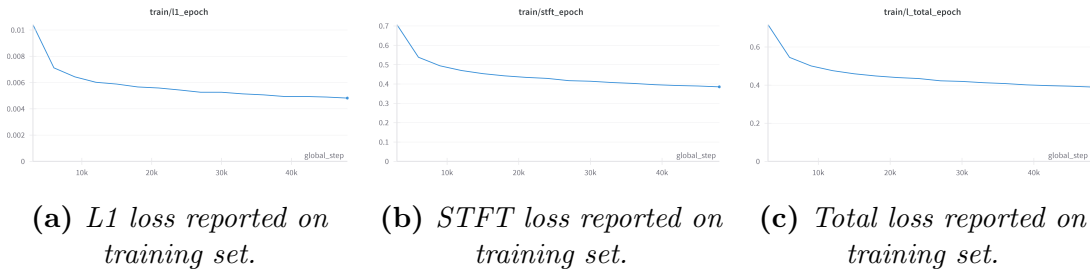


Figure 7.1: Training logs for CleanUNet model.

Metric	Reported	Reproduced
STOI	0.977	0.9319
PESQ-WB	3.146	2.029
PESQ-NB	3.551	2.582

Table 7.1: Metrics on test set of DNS-2020 dataset (without reverb) for CleanUNet model.

7.2.2 Denoiser

We tried to replicate the training procedure stated in [3] for the Denoiser model based on DEMUCS architecture [38]. We used 100 hours of the DNS 2020 dataset [29] for training, and its corresponding test set was used for testing.

Architecture: The depth of the encoder or the decoder is $D = 5$. Each encoder/decoder layer has hidden dimension $H = 48$, stride $S = 4$, and kernel size $K = 8$. These parameters were adapted from the original paper.

Optimization: The model was trained for 28K steps with a batch size of 14. The Adam [37] optimizer was used with a learning rate of 3×10^{-4} with a Linear Warmup Cosine Decay and beta values of 0.9 and 0.999. The loss functions used were L1 loss and Multi-Resolution STFT loss. The model was trained on 2 NVIDIA Tesla T4 GPUs. The training took 24 hours to complete. The training loss is shown in Figure 7.2 and metrics on the test set are shown in Table 7.2.

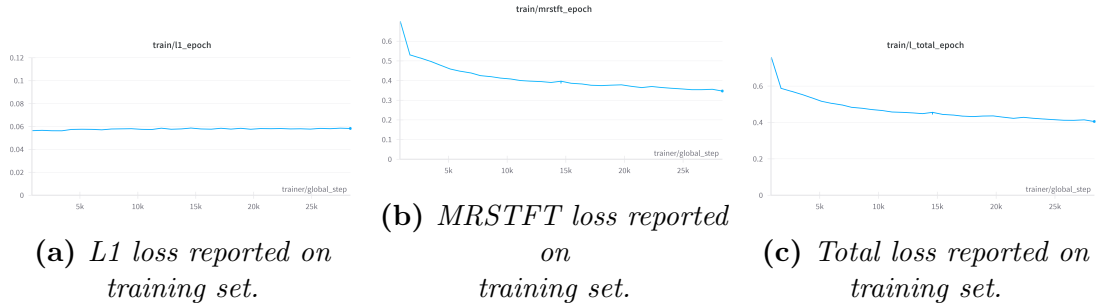


Figure 7.2: Training logs for Denoiser model.

Metric	Reported	Reproduced
STOI	0.95	0.934
PESQ-WB	3.07	2.129
PESQ-NB	—	2.641
SIG (MOS)	—	3.45
BAK (MOS)	—	4.023
OVRL (MOS)	—	3.172

Table 7.2: Metrics on test set of DNS-2020 dataset (without reverb) for Denoiser model.

7.2.3 VoiceFilter

The experiments for VoiceFilter [40] started before the completion of the project structure. Therefore, VoiceFilter experiments were conducted outside the project structure. We used an unofficial implementation of VoiceFilter available on GitHub: <https://github.com/maum-ai/voicefilter>. Although the implementation is unofficial and not provided by Google, the authors of the paper, it was acknowledged by the authors in the latest version of the paper.

Unfortunately, we couldn't reproduce the results of the paper. Our experiments lasted 108 hours. We operated on the same subset of LibriSpeech dataset [33] as the paper. We trained the model using a single P100 GPU.

in the output layer. The output of the sigmoid activation is the probability of the speaker’s voice residing in the subband.

7.3.3 Denoising Module

The denoising module is a regular FFSN model in which its decoder input subbands are scaled based on the speaker embedding module’s output. Subbands in which the speaker’s voice is detected are amplified, while the others are suppressed. This process aims to help the decoder focus on enhancing the speaker’s voice while suppressing the noise.

7.3.4 Experimental Setup

We trained the PFFSN model on the training set of our dataset for 43 epochs with a batch size of 16 per GPU. We trained the model using Kaggle’s 2 NVIDIA Tesla T4 GPUs. The noisy audio clips were generated on the fly by mixing speakerA and speakerB clean audio clips. The SNR of the noisy audio clips was randomly selected from values of 0, 3, 6, 10, 20. The clean audio clips were cropped to 3 seconds before mixing as an augmentation technique. The audio clips were sampled at 16kHz. The Adam[37] optimizer was used with a learning rate of 5×10^{-5} with Cosine Annealing[58] and beta values of 0.9 and 0.999. Model exponential moving average (EMA) was used with a decay of 0.999. We employed a mixture of loss functions, including L1 loss, Multi-Resolution STFT loss[3], complex Ration Mask (cRM) loss[2], and TAP loss[59] with weights 1, 0.5, 1, and 0.3 respectively. The weights of the Speaker Embedding Module and Denoising Module were initialized using weights from an FFSN model trained on DNS 2021 data to overcome limited data availability. Figure 7.4 shows the training loss curve of the PFFSN model.

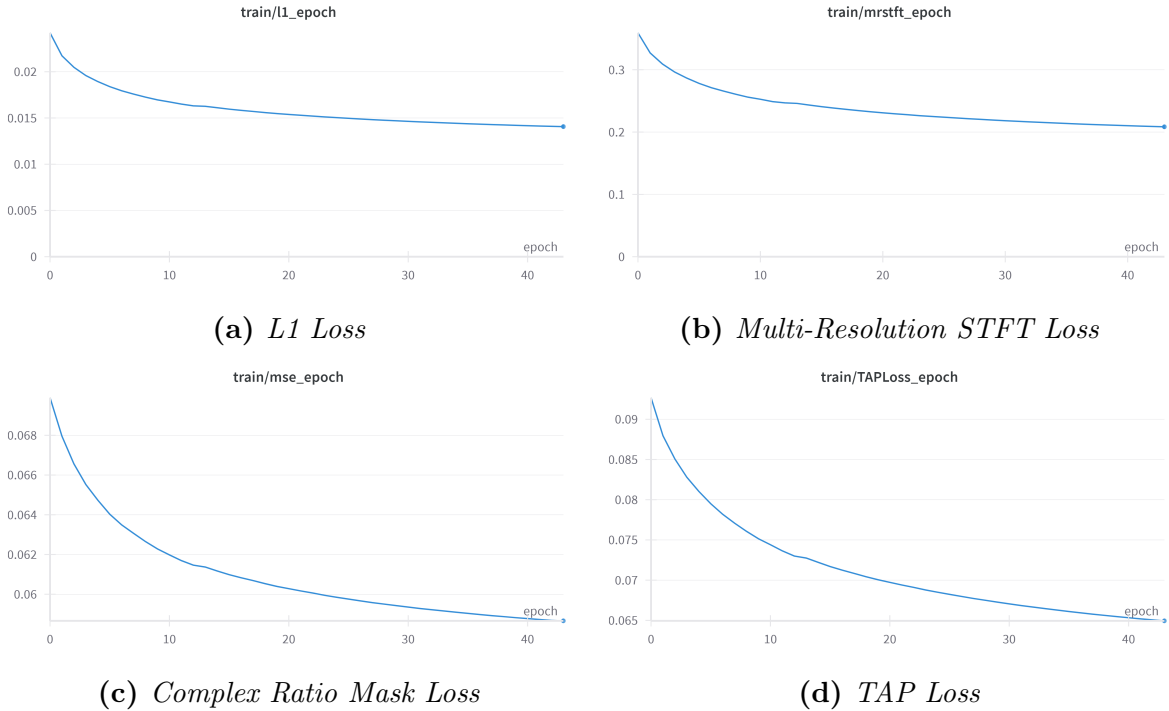


Figure 7.4: PFFSN model training loss curves

Metric	Collected DataSet (Different SNR)					Personalized DNS
	0	3	6	10	20	Blind set
DNSMOS/SIG	3.291	3.312	3.362	3.533	3.781	3.524
DNSMOS/BAK	1.848	2.004	2.252	2.661	3.393	2.824
DNSMOS/OVRL	1.963	2.080	2.263	2.587	3.113	2.518
PESQ nb	1.468	1.741	2.075	2.512	3.253	-
PESQ wb	1.205	1.380	1.654	2.086	2.927	-
STOI	0.717	0.804	0.866	0.916	0.955	-

Table 7.3: *PFFSN model results on the collected dataset test set using different SNR levels and on the personalized DNS blind test set*

7.3.5 Results

We tested the model on two test sets: a personalized DNS blind test set and the test set of our dataset. Results are shown in table 7.3.

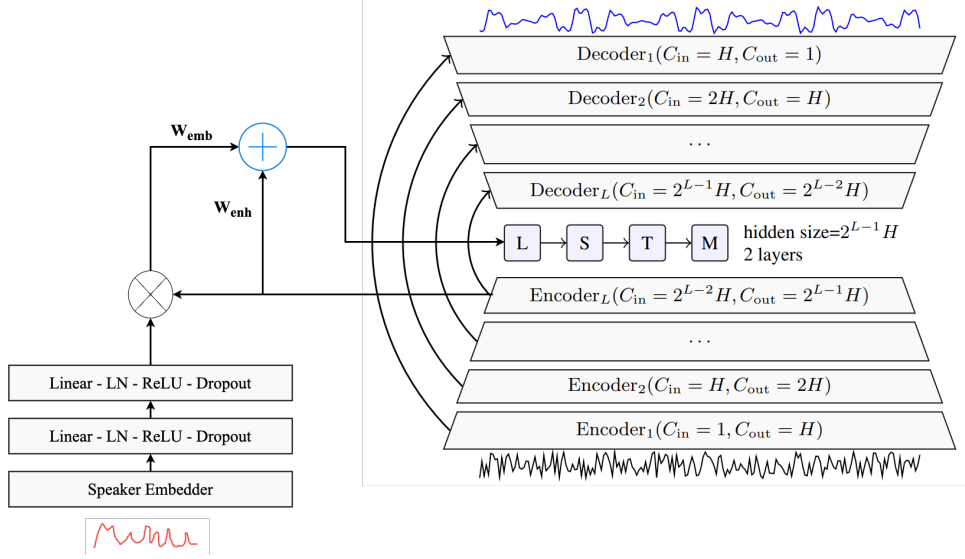
7.3.6 Limitations

After listening to some outputs, one can notice that the performance of the model degrades when the tones of the voice in the input change frequently. This is due that the model follows the subband approach that focuses on the removal of stationary noises. One can also notice that the model performs well when the interfering voice or noises reside in frequency ranges different from the ones where the main voice lies e.g., male and female interference, due to the ease of subband classification. On the other hand, the model finds hard times when the voices are in the same ranges e.g., male and male interference. Furthermore, low SNR audio clips are harder to enhance, low SNR audio might simulate scenarios where the two speakers are within the same proximity to the microphone which might not be the case in a teleconferencing scenario.

7.4 Personalized Denoiser

7.4.1 Motivation

The Denoiser model [3] offers raw waveform speech enhancement using an encoder-decoder architecture with a U-Net structure. Although the model great performance, it lacks the ability to personalize the enhancement process. To introduce personalization, we propose a personalized denoiser model or simply PDenoiser. PDenoiser consists of two major modules: a speaker embedding module and a denoising module. The speaker embedding module is responsible for extracting speaker-specific features from the input speech signal, while the denoising module is responsible for enhancing the speech signal based on the extracted speaker features. The architecture of the PDenoiser model is shown in Figure 7.5.

Figure 7.5: *PDenoiser Architecture*

7.4.2 Speaker Embedding Module

We primarily utilize TitaNet [48] to extract speaker embeddings from input speech signals of size 192. These speaker embeddings are then processed through a block, repeated twice, comprising a linear layer, followed by layer normalization, a ReLU activation function, and a dropout layer with a rate of 0.4. This block serves to enhance the dimensionality of the speaker embeddings to 768.

7.4.3 Denoising Module

The denoising module employs a standard Denoiser model. The output of encoder is reweighted by embedding module output. To ease gradient flow, a weighted residual connection is added from the encoder output. The weighted residual connection forms a convex linear combination with parameter $W_{enh} = 0.999$.

7.4.4 Experimental Setup

We trained the PDenoiser model on the training set of our dataset for 12 epochs with a batch size of 32 per GPU. We trained the model using Kaggle’s 2 NVIDIA Tesla T4 GPUs. The noisy audio clips were generated on the fly by mixing speakerA and speakerB clean audio clips. The SNR of the noisy audio clips was randomly selected from values of 0, 3, 6, 10 and 20. The clean audio clips were cropped to 3 seconds before mixing as an augmentation technique. The audio clips were sampled at 16kHz. The Adam optimizer was used with a learning rate of 1×10^{-4} with Cosine Annealing [58] and beta values of 0.9 and 0.999. Model exponential moving average (EMA) was used with a decay of 0.999. We employed a mixture of loss functions, including L1 loss, Multi Resolution STFT loss and TAP loss [59] with weights 1, 0.4 and 0.3 respectively. TitaNet was frozen during the training of the PDenoiser model. For the speaker denoising module, we utilized the pretrained weights from Meta’s Denoiser model, specifically the one with 48 hidden channels H , pre-trained on the DNS dataset. Figure 7.6 illustrates the training loss curves of the PDenoiser model for the two variations.

Metric	Collected DataSet (Different SNR)					Personalized DNS
	0	3	6	10	20	Blind set
DNSMOS/SIG	4.108	3.869	3.716	3.887	4.025	3.748
DNSMOS/BAK	1.725	2.248	3.475	3.874	3.988	2.824
DNSMOS/OVRL	2.183	2.458	3.086	3.413	3.575	2.711
PESQ nb	1.429	1.891	2.431	2.830	3.345	-
PESQ wb	1.171	1.495	1.998	2.423	3.038	-
STOI	0.690	0.849	0.913	0.941	0.962	-

Table 7.4: *PDenoiser model results on the collected dataset test set using different SNR levels and on the personalized DNS blind test set*

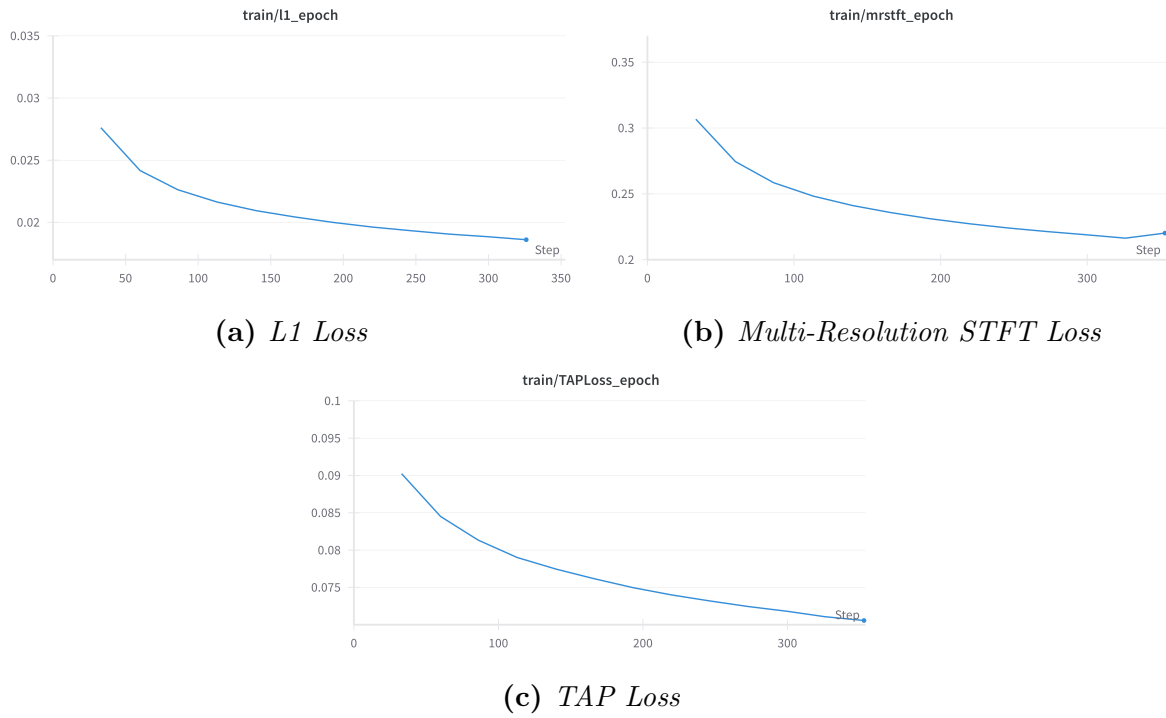


Figure 7.6: *PDenoiser model training loss curves*

7.4.5 Results

We tested the model on two test sets: personalized DNS blind test set and the test set of our dataset. Results are show in table 7.4.

7.4.6 Limitations

Upon listening to the outputs of the PDenoiser model on our dataset, we noticed that it struggles when the background speaker(s) noise is loud. In general, background speaker(s) audio is not fully eliminated but is suppressed. Additionally, the model size is large compared to the Personalized FFSN model.

Model	SIG MOS	BAK MOS	OVRL MOS
Team36_UnbeatableTencent_p_Pref1	3.64	2.92	2.72
PDenoiser	3.748	2.824	2.711
Team14_TencentASSP_p_Pref1	3.60	2.84	2.65
Team13_TSpeech-AI_Pref1	3.64	2.88	2.66
Team18_TencentVPPaaS_Pref1	3.61	2.86	2.64
Team34_NAPSE_p_Pref1	3.60	2.78	2.58
Personalized Fast FullSubNet	3.52	2.82	2.52
Team9_NJUAALab2_nonp_Pref1	3.54	2.77	2.54
Team40_SZAudio_p_Pref1	3.53	2.58	2.39
Baseline_p	3.22	2.68	2.38
Noisy	3.83	1.22	1.24

Table 7.5: *Personalized P.835 MOS results on DNS2023 blind test set. The models started by **Team** are the top 7 models submitted to the DNS2023 challenge. The **Baseline** model is the baseline model provided by the challenge organizers. The **Noisy** row represents the noisy audio clips.*

Model	DNSMOS SIG	DNSMOS BAK	DNSMOS OVRL	PESQ nb	PESQ wb	STOI
Personalized Fast FullSubNet	3.362	2.252	2.263	2.075	1.654	0.866
PDenoiser	3.748	2.824	2.711	2.431	1.998	0.913

Table 7.6: *Results of trained models on our collected test set at an SNR of 6.*

7.5 Model Selection

To assess the performance of our models we compared the results of our models against the top 7 models submitted to the DNS2023 challenge and the baseline model provided by the challenge organizers. Table 7.5 shows the results of the models on the DNS2023 blind test set and their ranking among other models based on OVRL MOS. Despite the limited data that our models were trained on compared to the other models, our models achieved competitive results. The Personalized Fast FullSubNet model ranked in the 6th place and PDenoiser got the 2nd place. This result shows that our models and cleaner dataset are capable to compete with models trained on much larger datasets.

For further evaluation, we compared the results of our models on our collected test set at an SNR of 6. The results are shown in table 7.6. The Personalized Denoiser model outperformed the Personalized Fast FullSubNet model in all the metrics.

Based on these results we decided to select the Personalized Denoiser model as our final model for the project. The model will be integrated into the application for a lightweight, personalized and real-time noise suppression solution.

Chapter 8

Application

In this section, we provide an overview of the application, detailing its primary components and functionality. The app is designed to enhance user interaction through a graphical user interface (GUI), stream audio input for real-time processing, and apply our machine learning model to process the streamed audio. The application workflow is divided into two main stages before reaching the ML model: the GUI app and the streamer.

The first stage involves the GUI app, which functions as the user interface for interaction. Users input their data through this intuitive and user-friendly interface. The GUI is responsible for capturing user inputs and passing them to the subsequent stages of the application. It ensures that users can easily and effectively interact with the system, providing the necessary inputs required for further processing.

The second stage involves the streamer, which is responsible for processing real-time audio input. Its main function is to capture the user's voice. The streamer is designed to effectively handle and stream the audio data, ensuring continuous input into the system without any interruptions. This real-time streaming capability is essential for applications that require immediate audio processing and analysis.

To locate the application, please follow this link: [targeted-speech-enhancement-app](#). You can find the application there, along with the necessary steps to run it.

The following is an in-depth explanation of the streamer in our application, along with its key features.

8.1 App Features

In the process of developing the application, we made several key design decisions to ensure efficiency, ease of use, and robust functionality. This subsection outlines the libraries and tools chosen for the graphical user interface (GUI) and database management, and presents the technical details and components of our user interface.

8.1.1 Graphical User Interface: Tkinter and CustomTkinter

The application's graphical user interface (GUI) has been meticulously crafted using the powerful **Tkinter** library, a standard Python interface to the Tk GUI toolkit. **Tkinter** is widely recognized for its simplicity and seamless integration with Python applications, offering an object-oriented interface that simplifies the development of responsive GUI applications.

In addition to **Tkinter**, the application leverages the enhanced capabilities of **CustomTkinter** to further elevate its visual appeal and functionality. **CustomTkinter** extends the features of **Tkinter** by providing advanced customization and theming options, including support for color modes, stylish switches, and dynamic menus. Through the seamless integration of **Tkinter** and **CustomTkinter**, the application not only delivers robust performance but also boasts a modern, user-friendly interface that enhances the overall user experience.

8.1.2 Database Management: SQLite

The application utilizes SQLite, a lightweight and self-contained SQL database engine, for database management. SQLite is integrated into Python through the **sqlite3** module, which offers a straightforward interface for database operations. We opted for SQLite due to its reliability, serverless nature, and zero-configuration database solution, making it ideal for our application.

SQLite's simplicity and ease of organization are its primary advantages. It stores the entire database in a single file on the disk, ensuring high portability and easy management. Given that our stored data isn't highly complex, SQLite was the perfect choice for us.

In Python, the **sqlite3** module enables efficient execution of SQL queries, management of database connections, and handling of transactions. This makes SQLite an optimal choice for applications that require a robust database solution without the overhead of a full-fledged database server.

By leveraging **Tkinter**, **CustomTkinter**, and SQLite, the application attains a harmonious blend of functionality, performance, and user experience.

8.1.3 Technical Details

The app is designed to enhance speech using the user's reference audio. Upon first use, the client must provide their name and upload a clear voice audio from their local device.

Afterward, the client needs to select the input device for real-time voice enhancement.

Additionally, the client can adjust the percentage of noise to be removed using the dry/wet slider.

Our app supports multiple users, each mapped to one reference audio. If a user needs to replace their reference audio, they can upload a new audio file, which will replace the old one.

Once these settings have been configured, the user will be able to activate the 'Cancel Noise' switch to start noise cancellation. Switching on disables other components, as the user is not allowed to modify their values while the app is running.

If the user deactivate the 'Cancel Noise' switch, the components are enabled again and noise cancellation is halted until the user edits their necessary data and clicks activate the switch again to resume.

Finally, the app is available in both light and dark modes to suit user preferences.

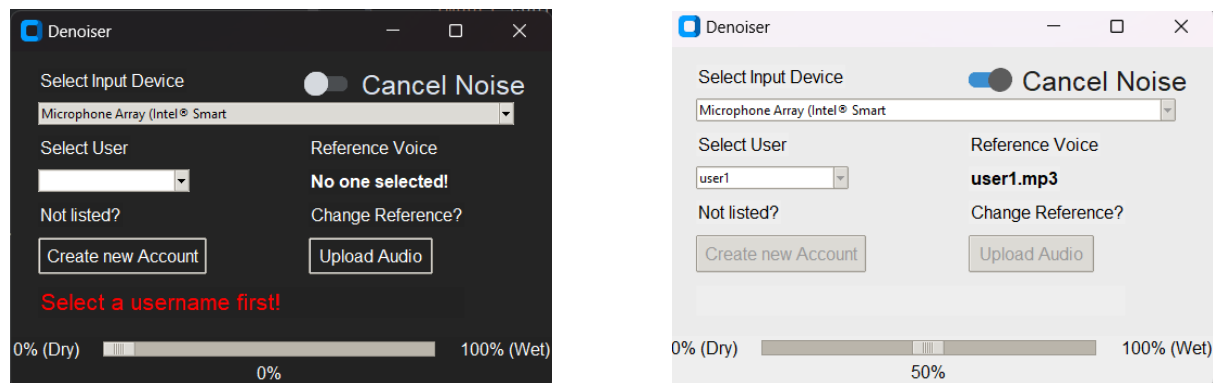


Figure 8.1: *Figure showing the appearance of our GUI in both: dark & light mode*

Here is a recap of the features or components provided by our GUI:

- **Select Input Device:** Dropdown menu that displays current audio inputs from the device.
- **Select User:** A dropdown menu that displays current saved usernames on the device. Once a user is selected, their reference voice's name appears next to the menu in bold.
- **Reference Voice:** Label that displays the user's reference audio once a user is chosen from the 'Select User' dropdown.
- **Create New Account:** Button that leads to another window, allowing the user to provide their name and upload their reference audio for the first time. Once this data is submitted, the user's name and reference audio remain saved in the app.
- **Upload Audio:** Button that enables a user to change their stored reference audio by uploading a new one.
- **Dry/Wet Slider:** Controls the amount of noise to keep or remove from the provided audio.
- **Cancel Noise:** Switch button that can be toggled on and off, initial state is 'off' while turning it on passes the given values to the model to start working.

8.2 Streaming

The streaming module is responsible for capturing real-time audio input from the user and feeding it into the machine-learning model for processing. It is designed to efficiently manage and stream the audio data, ensuring uninterrupted feeding into the system. This

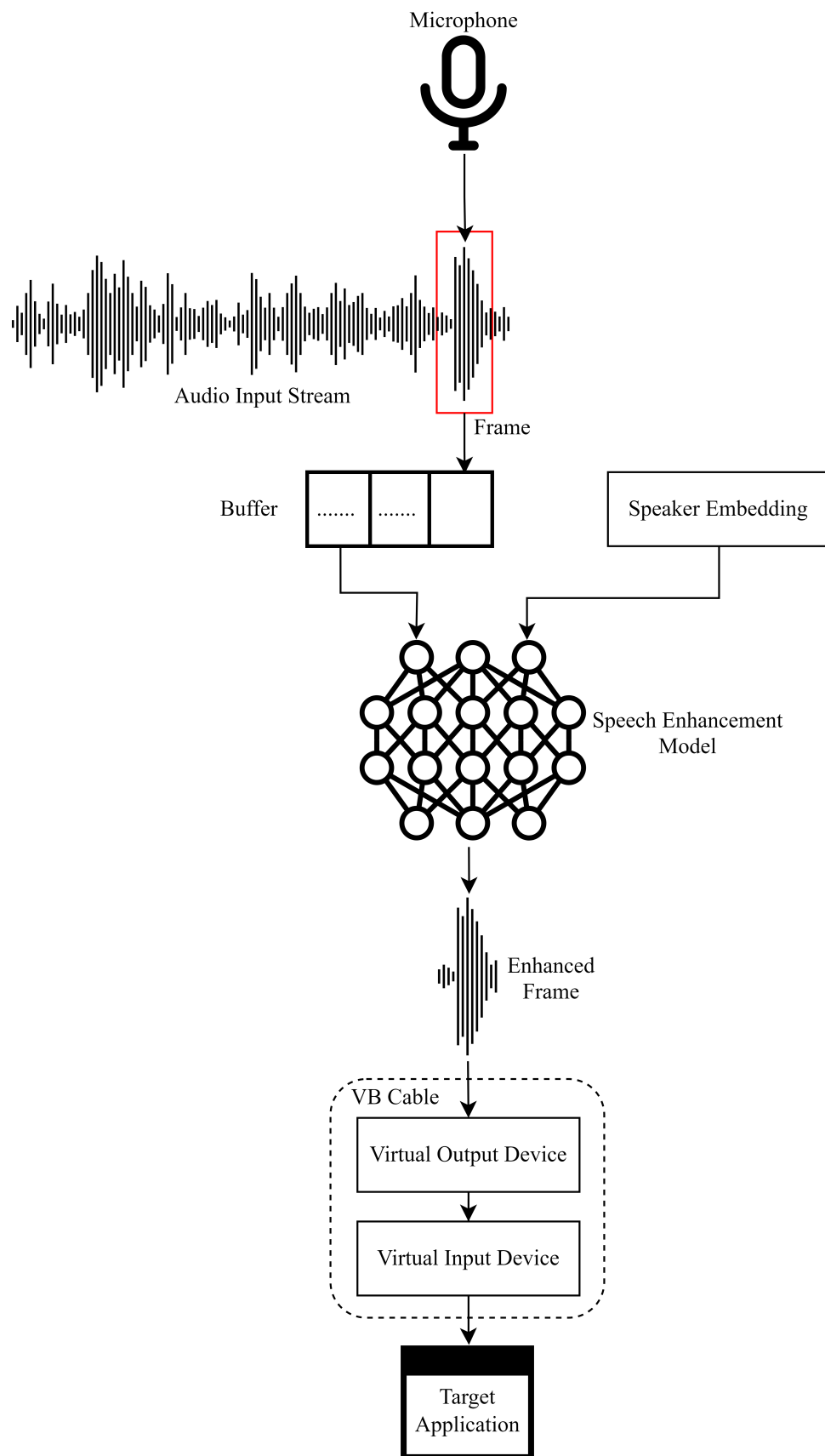


Figure 8.2: *Streaming pipeline*

section provides an overview of the streaming module, detailing its primary components and functionality.

The streaming pipeline begins with the user’s microphone capturing audio input. The streaming module gathers small chunks of audio data, known as frames, from the microphone. Typically, the frame size is set to 60 milliseconds, but this can be adjusted based on the specific application requirements. There’s a trade-off between frame size, latency, and system performance. A smaller frame size reduces latency, while longer frames offer better audio quality processed by the model due to the increased context but result in higher latency.

The collected audio frames are continually pushed into a buffer, which is a queue-like data structure that maintains a first-in-first-out (FIFO) order. This ensures that the audio frames are processed in the order they were received. Concurrently, the streaming module continuously feeds the audio frames into the machine-learning model for processing. The enhanced audio frames are then sent to the virtual output device, which routes the processed audio frames to the virtual microphone. This virtual microphone is utilized by the user’s applications as the input device. You can refer to Figure 8.2 for an illustration of the streaming pipeline.

In order to improve the quality of the enhanced audio, we have included the ability to process multiple audio frames simultaneously. This gives the model more context to work with, resulting in higher audio quality. However, this may lead to increased latency, as the output device will need to wait until a certain number of full frames are collected. This will result in the following latency:

$$\text{latency} = n \times \text{frame length} + n \times \text{processing time per frame}$$

To mitigate this, instead of keeping the streaming module idle while waiting for the model to process the audio frames, we start by processing the first frame and then save this frame for future processing. The $n - 1$ saved past frames are then concatenated to the new frame to be processed by the model. This way, the streaming module can continue to collect and process audio frames without waiting for the full n frames to arrive making the overall latency:

$$\text{latency} = \text{frame length} + n \times \text{processing time}$$

This allows the streaming module to keep up with the real-time audio input and reduce the overall latency of the system. However, this approach may introduce a delay due to the added processing time of the model. This added delay is way less than the latency introduced by waiting for the full n frames to arrive since the processing time is typically less than the frame length to achieve real-time constraints. We also added the option to adjust the number of working threads to process the audio frames. This far reduces the processing time of the model, which results in lower latency. If the user set the number of threads to n , the model will process n frames at the same time, which will result in a latency of almost:

$$\text{latency} = \text{frame length} + \text{processing time}$$

This will result in the lowest latency possible, almost the same as processing 1 frame at a time. However, it will require more processing power.

In our experiments, we set the number of frames n to 2 and the number of threads to 2. This configuration resulted in a low RTF of 0.21, which is excellent for real-time

applications with relatively low CPU utilization of 20%. This experiment was conducted on a laptop with a CPU AMD Ryzen 9 5900HX. We can further reduce the latency and RTF by using GPU acceleration.

Chapter 9

Future Work

This chapter discusses the potential future work and improvements that can be made to the project.

1. Increasing Data Size:

- Enhance the dataset quality by incorporating additional speakers and a wider range of languages.

2. Data Synthesis:

- Synthesize data with varying noise levels and diverse noise types to increase the model's robustness.

3. Handling Noisy Reference Audio:

- Develop strategies to make the model resilient to noisy reference audio inputs.

4. Different Sampling Rates Support:

- The suggested models are currently designed to work with audio sampled at 16kHz. Extend support to include other sampling rates such as 44.1 kHz and 48 kHz.

5. Operating System Support:

- Extend support to include additional operating systems such as Linux.

References

- [1] H. Dubey and et al., “Icassp 2023 deep noise suppression challenge,” in *2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023. DOI: 10.48550/arXiv.2303.11510.
- [2] X. Hao and X. Li, *Fast fullsubnet: Accelerate full-band and sub-band fusion model for single-channel speech enhancement*, 2023. arXiv: 2212.09019.
- [3] A. Defossez, G. Synnaeve, and Y. Adi, *Real time speech enhancement in the waveform domain*, 2020. arXiv: 2006.12847 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2006.12847>.
- [4] A. Mehrish, N. Majumder, R. Bhardwaj, R. Mihalcea, and S. Poria, “A review of deep learning techniques for speech processing,” *arXiv.org*, May 2023. eprint: 2305.00359. [Online]. Available: <https://arxiv.org/abs/2305.00359>.
- [5] C. K. A. Reddy, N. Shankar, G. Bhat, R. Charan, and I. Panahi, “An individualized super gaussian single microphone speech enhancement for hearing aid users with smartphone as an assistive device,” *arXiv*, Dec. 2018. eprint: cs.SD.
- [6] C. K. A. Reddy, E. Beyrami, J. Pool, R. Cutler, S. Srinivasan, and J. Gehrke, “A scalable noisy speech dataset and online subjective test framework,” *arXiv*, Sep. 2019. eprint: cs.SD.
- [7] K. Lebart, J. Boucher, and P. N. Denbigh, “A new method based on spectral subtraction for speech dereverberation,” *Acta Acustica United With Acustica*, vol. 87, p. 359, May 2001.
- [8] E. Vincent, T. Virtanen, and S. Gannot, *Audio Source Separation and Speech Enhancement*. Hoboken, NJ: John Wiley & Sons, Inc., 2018.
- [9] E. Cano, D. FitzGerald, A. Liutkus, M. D. Plumbley, and F.-R. Stoter, “Musical source separation: An introduction,” *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 31–40, 2019.
- [10] Z. Li and M.-W. Mak, *Speaker representation learning via contrastive loss with maximal speaker separability*, 2022. arXiv: 2210.16636 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2210.16636>.
- [11] Y. Kwon, J.-w. Jung, H.-S. Heo, Y. J. Kim, B.-J. Lee, and J. S. Chung, *Adapting speaker embeddings for speaker diarisation*, 2021. arXiv: 2104.02879 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2104.02879>.
- [12] T. Bäckström, O. Räsänen, A. Zewoudie, et al., *Introduction to Speech Processing*, 2nd ed. 2022. DOI: 10.5281/zenodo.6821775. [Online]. Available: <https://speechprocessingbook.aalto.fi>.
- [13] N. Mesgarani and E. F. Chang, “Selective cortical representation of attended speaker in multi-talker speech perception,” *Nature*, vol. 485, no. 7397, pp. 233–236, 2012.

- [14] J. Agrawal, M. Gupta, and H. Garg, “A review on speech separation in cocktail party environment: Challenges and approaches,” *Multimedia Tools and Applications*, vol. 82, no. 20, pp. 31 035–31 067, Feb. 2023. DOI: 10.1007/s11042-023-14649-x.
- [15] J. Agrawal, M. Gupta, and H. Garg, “Monaural speech separation using wt-conv-tasnet for hearing aids,” *International Journal of Speech Technology*, vol. 26, no. 3, pp. 707–720, Oct. 2023. DOI: 10.1007/s10772-023-10045-w.
- [16] S. M. Town and J. K. Bizley, “Neural and behavioral investigations into timbre perception,” *Frontiers in Systems Neuroscience*, vol. 7, Nov. 2013. DOI: 10.3389/fnsys.2013.00088.
- [17] J. Y. Lee and et al., “Speech recognition in real-life background noise by young and middle-aged adults with normal hearing,” *Journal of Audiology and Otology*, vol. 19, no. 1, pp. 39–44, 2015. DOI: 10.7874/jao.2015.19.1.39.
- [18] K. Zmolikova and et al., “Neural target speech extraction: An overview,” *IEEE Signal Processing Magazine*, vol. 40, no. 3, pp. 8–29, 2023. DOI: 10.1109/msp.2023.3240008.
- [19] A. W. Bronkhorst, “The cocktail-party problem revisited: Early processing and selection of multi-talker speech,” *Attention, Perception, Psychophysics*, vol. 77, no. 5, pp. 1465–1487, 2015. DOI: 10.3758/s13414-015-0882-9.
- [20] P. S. and J. S. E. Manilow. “Open source tool and data for music source separation.” (Oct. 2020), [Online]. Available: <https://source-separation.github.io/tutorial>.
- [21] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” *ArXiv e-prints*, Mar. 2016. eprint: 1603.07285.
- [22] C. Olah. “Understanding LSTM networks.” (Aug. 2015), [Online]. Available: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- [23] E. M., P. S., and J. Salamon. “Open source separation.” (Oct. 2020), [Online]. Available: <https://source-separation.github.io/tutorial>.
- [24] S. Hochreiter. “Long short-term memory.” (1997), [Online]. Available: <https://www.bioinf.jku.at/publications/older/2604.pdf>.
- [25] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>.
- [26] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” *arXiv preprint arXiv:1505.04597*, 2015. [Online]. Available: <https://arxiv.org/abs/1505.04597>.
- [27] C. Valentini-Botinhao, X. Wang, S. Takaki, and J. Yamagishi, “Speech enhancement for a noise-robust text-to-speech synthesis system using deep recurrent neural networks,” in *Interspeech 2016*, 2016. DOI: 10.21437/interspeech.2016-159.
- [28] J. Thiemann, N. Ito, and E. Vincent, “The diverse environments multi-channel acoustic noise database (demand): A database of multichannel environmental noise recordings,” in *Montreal, Canada*, 2013, pp. 035 081–035 081. DOI: 10.1121/1.4799597.
- [29] C. K. A. Reddy and et al., “The interspeech 2020 deep noise suppression challenge: Datasets, subjective testing framework, and challenge results,” in *Interspeech 2020*, 2020. DOI: 10.21437/interspeech.2020-3038.

- [30] “Icassp 2022 deep noise suppression challenge,” in *2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022. DOI: 10.48550/arXiv.2202.13288.
- [31] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015. DOI: 10.1109/icassp.2015.7178964.
- [32] J. F. Gemmeke and et al., “Audio set: An ontology and human-labeled dataset for audio events,” in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017. DOI: 10.1109/icassp.2017.7952261.
- [33] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210. DOI: 10.1109/ICASSP.2015.7178964.
- [34] C. K. A. Reddy, V. Gopal, and R. Cutler, *Dnsmos: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors*, 2021. arXiv: 2010.15258 [cs.SD]. [Online]. Available: <https://arxiv.org/abs/2010.15258>.
- [35] C. K. A. Reddy, V. Gopal, and R. Cutler, *Dnsmos p.835: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors*, 2022. arXiv: 2110.01763 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2110.01763>.
- [36] C. K. A. Reddy, H. Dubey, K. Koishida, et al., *Interspeech 2021 deep noise suppression challenge*, 2021. arXiv: 2101.01902 [cs.SD]. [Online]. Available: <https://arxiv.org/abs/2101.01902>.
- [37] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, 2017. arXiv: 1412.6980 [cs.LG].
- [38] A. Défossez, N. Usunier, L. Bottou, and F. R. Bach, “Music source separation in the waveform domain,” *CoRR*, vol. abs/1911.13254, 2019. arXiv: 1911.13254. [Online]. Available: <http://arxiv.org/abs/1911.13254>.
- [39] Z. Kong, W. Ping, A. Dantrey, and B. Catanzaro, “Speech denoising in the waveform domain with self-attention,” in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2022.
- [40] Q. Wang, H. Muckenhirn, K. Wilson, et al., *Voicefilter: Targeted voice separation by speaker-conditioned spectrogram masking*, 2019. arXiv: 1810.04826 [eess.AS].
- [41] L. Wan, Q. Wang, A. Papir, and I. L. Moreno, *Generalized end-to-end loss for speaker verification*, 2020. arXiv: 1710.10467 [eess.AS].
- [42] K. W. Wilson, M. Chinen, J. Thorpe, et al., “Exploring tradeoffs in models for low-latency speech enhancement,” *CoRR*, vol. abs/1811.07030, 2018. arXiv: 1811.07030. [Online]. Available: <http://arxiv.org/abs/1811.07030>.
- [43] Q. Wang, I. L. Moreno, M. Saglam, et al., *Voicefilter-lite: Streaming targeted voice separation for on-device speech recognition*, 2020. arXiv: 2009.04323 [eess.AS].
- [44] J. Chen, W. Rao, Z. Wang, et al., *Inter-subnet: Speech enhancement with subband interaction*, 2023. arXiv: 2305.05599 [cs.SD].
- [45] X. Hao, X. Su, R. Horaud, and X. Li, “Fullsubnet: A full-band and sub-band fusion model for real-time single-channel speech enhancement,” in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 6633–6637. DOI: 10.1109/ICASSP39728.2021.9414177.
- [46] Y. Wu and K. He, *Group normalization*, 2018. arXiv: 1803.08494 [cs.CV].

- [47] Y. Luo, Z. Chen, N. Mesgarani, and T. Yoshioka, *End-to-end microphone permutation and number invariant multi-channel speech separation*, 2020. arXiv: 1910.14104 [eess.AS].
- [48] N. R. Koluguri, T. Park, and B. Ginsburg, *Titanet: Neural model for speaker representation with 1d depth-wise separable convolutions and global context*, 2021. arXiv: 2110.04410 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2110.04410>.
- [49] A. Nagrani, J. S. Chung, and A. Zisserman, *Voxceleb: A large-scale speaker identification dataset*, Aug. 2017. DOI: 10.21437/interspeech.2017-950. [Online]. Available: <http://dx.doi.org/10.21437/Interspeech.2017-950>.
- [50] J. S. Chung, A. Nagrani, and A. Zisserman, *Voxceleb2: Deep speaker recognition*, Sep. 2018. DOI: 10.21437/interspeech.2018-1929. [Online]. Available: <http://dx.doi.org/10.21437/Interspeech.2018-1929>.
- [51] B. Desplanques, J. Thienpondt, and K. Demuynck, *Ecapa-tdnn: Emphasized channel attention, propagation and aggregation in tdnn based speaker verification*, Oct. 2020. DOI: 10.21437/interspeech.2020-2650. [Online]. Available: <http://dx.doi.org/10.21437/Interspeech.2020-2650>.
- [52] J. S. Chung, A. Nagrani, E. Coto, *et al.*, *Voxsrc 2019: The first voxceleb speaker recognition challenge*, 2019. arXiv: 1912.02522 [cs.SD]. [Online]. Available: <https://arxiv.org/abs/1912.02522>.
- [53] Y. Ju, J. Chen, S. Zhang, *et al.*, *Tea-pse 3.0: Tencent-ethereal-audio-lab personalized speech enhancement system for icassp 2023 dns challenge*, 2023. arXiv: 2303.07704 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2303.07704>.
- [54] S. Gao, M. Cheng, K. Zhao, X. Zhang, M. Yang, and P. H. S. Torr, “Res2net: A new multi-scale backbone architecture,” *CoRR*, vol. abs/1904.01169, 2019. arXiv: 1904.01169. [Online]. Available: <http://arxiv.org/abs/1904.01169>.
- [55] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” *CoRR*, vol. abs/1709.01507, 2017. arXiv: 1709.01507. [Online]. Available: <http://arxiv.org/abs/1709.01507>.
- [56] D. Snyder, D. Garcia-Romero, G. Sell, A. McCree, D. Povey, and S. Khudanpur, *Speaker recognition for multi-speaker conversations using x-vectors*, 2019. DOI: 10.1109/ICASSP.2019.8683760.
- [57] D. Snyder, G. Chen, and D. Povey, “MUSAN: A music, speech, and noise corpus,” *CoRR*, vol. abs/1510.08484, 2015. arXiv: 1510.08484. [Online]. Available: <http://arxiv.org/abs/1510.08484>.
- [58] I. Loshchilov and F. Hutter, *Sgdr: Stochastic gradient descent with warm restarts*, 2017. arXiv: 1608.03983 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1608.03983>.
- [59] Y. Zeng, J. Konan, S. Han, *et al.*, *Taploss: A temporal acoustic parameter loss for speech enhancement*, 2023. arXiv: 2302.08088 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2302.08088>.